

Badge reader

Dossier de Projet CDA
Présentation et Défense

Informations du Projet

Candidat : Bernard Théoden

Promotion : 2023/2026

Soutenance : [Date]

Ce dossier présente le projet réalisé dans le cadre de la formation
Concepteur Développeur d'Applications (CDA) de Colint School

Année académique 2025-2026

Table des matières

1	Présentation du projet	5
1.1	Introduction	5
1.2	Problématique et objectifs SMART	6
1.3	Liens utiles	6
2	Cadrage et cahier des charges	7
2.1	Objectifs métier, techniques et pédagogiques	7
2.2	Définition du MVP	8
2.3	Cibles et parties prenantes	9
2.4	Exigences fonctionnelles	9
2.4.1	Fonctionnalités « Front Office »	10
2.4.2	Fonctionnalités « Back Office »	10
2.4.3	Confidentialité et RGPD	10
2.4.4	Droits d'accès	10
2.4.5	Authentification	11
2.5	Exigences et choix techniques	11
2.5.1	Exigences non fonctionnelles	11
2.5.2	Choix technologiques	11
2.6	Roadmap	12
2.7	Liens utiles	13
3	Méthodologie et organisation	15
3.1	Gestion de projet avec GitHub	15
3.1.1	User Stories et estimation de temps	15
3.2	Versioning GitHub et conventions	16
3.3	Planification et outils de suivi	16
3.4	Estimation de charge et planification	17
4	Conception fonctionnelle et technique	19
4.1	Cas d'usage et modélisation UML	19
4.2	Diagrammes de séquence	20
4.3	Conception de l'interface utilisateur	21
4.3.1	Zoning	21
4.3.2	Wireframe	22
4.3.3	Maquettage	24
4.3.4	Outils de conception et diagrammes	26
4.3.5	Charte graphique	27
4.4	Conception de la base de données	27
4.4.1	MCD (Modèle Conceptuel de Données)	28
4.4.2	MLD (Modèle Logique de Données)	28
4.4.3	MPD (Modèle Physique de Données)	29
4.5	Architecture 3 tiers	30

4.6 Liens utiles	30
5 Architecture 3 tiers	31
5.1 Architecture 3 tiers	31
5.1.1 Couche Présentation (Frontend)	31
5.1.2 Couche Logique Métier (Backend)	31
5.1.3 Couche Données (Database)	33
5.1.4 Communication entre les tiers	33
5.1.5 Avantages de l'architecture 3 tiers	34
5.2 Développement Frontend	35
5.3 Développement Backend	36
5.4 Gestion des données	37
5.5 Liens utiles	39
6 Sécurité applicative et RGPD	41
6.1 Protection contre les vulnérabilités OWASP	41
6.2 Authentification et autorisation	43
6.3 Conformité RGPD	46
6.4 Liens utiles	49
7 Tests et qualité logicielle	51
7.1 Stratégie de tests	51
7.2 Tests de performance	53
7.3 Qualité du code avec SonarQube	55
7.4 Liens utiles	58
8 Déploiement et CI/CD	59
8.1 Containerisation avec Docker	59
8.2 Pipeline CI/CD avec GitHub Actions	60
8.3 Documentation et monitoring	62
8.4 Liens utiles	66
9 Veille technologique et sécurité	67
9.1 Veille technologique stack	67
9.2 Bonnes pratiques sécurité	68
9.3 Application au projet	69
9.4 Liens utiles	71
10 Bilan et retour d'expérience (REX)	73
10.1 Objectifs atteints et non atteints	73
10.2 Difficultés rencontrées et solutions	73
10.3 Dettes techniques et apprentissages	74
10.4 Liens utiles	76
11 Conclusion et remerciements	77
11.1 Synthèse du projet	77
11.2 Perspectives d'évolution	78
11.3 Remerciements	79
11.4 Déploiement et documentation	79
11.4.1 Docker	80
11.4.2 GitHub (code source)	81
11.4.3 CI/CD	82
11.4.4 SonarQube	82
11.4.5 Swagger	83
11.5 Liens utiles	85

Chapitre 1

Présentation du projet

1.1 Introduction

Ce dossier présente le projet *Badge reader*, un système de gestion de badges que j'ai conçu pour sécuriser et fluidifier les accès au bâtiment de Colint. En tant qu'étudiant en formation **Concepteur Développeur d'Applications (CDA)**, j'ai pu constater les limites du contrôle d'accès actuel, ce qui m'a motivé à développer cette solution.

Dans ce chapitre, je détaille mon rôle dans le développement d'un site web dédié à la gestion des badges, ainsi que l'organisation du projet et mes responsabilités. L'objectif est de répondre aux exigences du titre RNCP 37873, tout en proposant une solution adaptée aux besoins de l'école.

Contexte et organisation du projet :

Dans le cadre de mon **projet de fin d'études (PFE)**, je mène la conception et le développement du système *Badge reader*, de l'analyse des besoins à la réalisation finale. Ce projet s'adresse aux membres de Colint et, à terme, à d'autres interlocuteurs.

Le système repose sur l'attribution d'un **badge individuel** à chaque utilisateur, permettant de contrôler les entrées et sorties du bâtiment. Les accès sont définis par des **rôles** :

- Après 18h, les étudiants ne peuvent plus entrer (toute sortie est définitive).
- Le staff dispose d'un droit d'entrée jusqu'à 20h.

Le déploiement est prévu pour le **1^{er} septembre 2026**.

L'application sera développée en **Elixir** avec le framework **Phoenix**, pour une intégration cohérente avec l'intranet existant de l'école. L'organisation du projet s'appuie sur **GitHub Projects** et un **système Kanban** pour structurer les tâches et suivre leur progression.

1.2 Problématique et objectifs SMART

Problématique identifiée : Comment rendre l'accès au bâtiment de Colint **plus fluide pour les étudiants et le staff**, tout en garantissant sa **sécurité** et son **contrôle** ?

Bénéfices attendus :

- Fluidifier le trafic dans le bâtiment.
- Vérifier la légitimité des personnes entrant dans l'école.
- Sécuriser les accès en fonction des rôles et des horaires.

Objectifs SMART et planning :

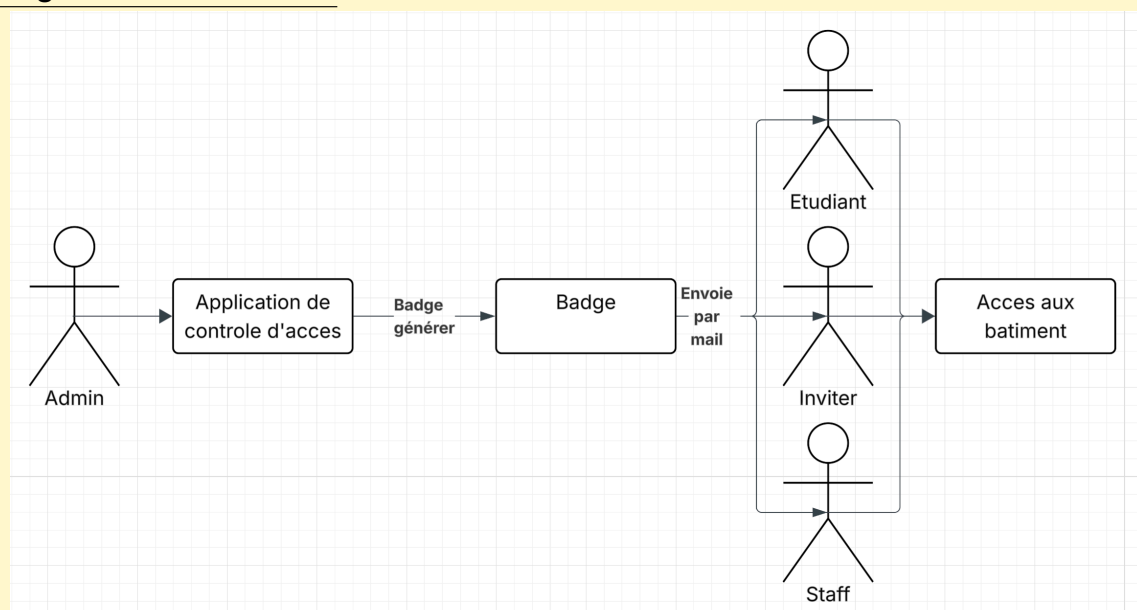
Phase 1 – Application principale :

- Développer une application pour **collecter et exploiter 100% des données** liées aux entrées/sorties.
- Gérer les utilisateurs, les rôles et les badges (physiques ou numériques).
- Livrer une **version 1.0 minimaliste et efficace** d'ici **mai 2026** (5 mois).

Phase 2 – Badge dématérialisé :

- Remplacer les badges physiques par une **solution mobile** (iOS/Android).
- Intégrer cette fonctionnalité dans une **version 1.1** en **2 mois**.

Diagramme de contexte :



1.3 Liens utiles

Les ressources ci-dessous m'ont servi de référence pour la méthodologie **SMART**, la gestion de projet et l'organisation du travail :

- Documentation GitHub : <https://docs.github.com/>
- Objectifs SMART (Atlassian) : <https://bit.ly/smart-goals-atlassian>
- Project Management Institute : <https://www.pmi.org/>
- Manifeste Agile : <https://agilemanifesto.org/>
- Business Model Canvas : <https://bit.ly/business-model-canvas>

Chapitre 2

Cadrage et cahier des charges

2.1 Objectifs métier, techniques et pédagogiques

Cette section définit les objectifs du projet selon trois axes : **métier**, **technique** et **pédagogique**. Ces objectifs guident le développement et garantissent la cohérence du système *Badge reader*.

3 grand objectifs :

Objectifs métier :

- Contrôler **100% des accès** au bâtiment, avec un suivi précis des heures d'entrée et de sortie.
- **Restreindre l'accès** aux seules personnes autorisées.

Objectifs techniques :

- Concevoir et développer une **application web** intégrant un système de badge et de contrôle d'accès.
- Mettre en œuvre des **mécanismes de sécurité robuste** : chiffrement des données, authentification par identifiant/mot de passe.

Objectifs pédagogiques :

- Concevoir et implémenter une **architecture logicielle en trois tiers**.
- Maîtriser les **bonnes pratiques GitHub** (gestion de versions, workflow, collaboration).
- Approfondir les **compétences en sécurité informatique** (authentification, protection des données).

2.2 Définition du MVP

Le **Minimum Viable Product (MVP)** regroupe les fonctionnalités essentielles pour valider l'hypothèse du projet. Il permet de :

- Vérifier la pertinence du système de contrôle d'accès.
- Recueillir des retours utilisateurs pour ajuster la feuille de route.

Tableau MoSCoW et fonctionnalités prioritaires :

Priorité	Fonctionnalité	Justification
Must Have	Application web	Indispensable : sans elle, le système ne peut pas fonctionner.
Must Have	Authentification et chiffrement	Critique : sécuriser les accès et les données.
Should Have	Génération et gestion des badges	Nécessaire pour un contrôle d'accès fiable.
Could Have	Connexion de l'API intranet au lecteur	Utile pour améliorer l'interopérabilité.
Won't Have	Intégration directe du lecteur dans l'intranet	Non prioritaire pour le MVP.

Scénarios essentiels du MVP : Le MVP devra permettre de :

- Gérer les badges des utilisateurs.
- Authentifier les accès et enregistrer les entrées/sorties.

Ces fonctionnalités valident le besoin métier et la faisabilité technique.

2.3 Cibles et parties prenantes

Nous allons identifier et analyser les utilisateurs cibles et de les categoriser. Chacun a des besoins spécifiques, détaillés ci-dessous :

Utilisateurs clés et leurs besoins :

- **Étudiants** : Accès du **lundi au vendredi (8h–17h)** pour les cours et projets.
Exemple : Un étudiant en retard doit pouvoir entrer rapidement.
- **Invités** : Badge **temporaire** (ex. : intervenant pour une journée). Droits limités à la durée de leur visite.
- **Staff** : Accès élargi (**lundi–samedi, 8h–19h**) pour les missions pédagogiques.
Exemple : Un formateur qui prépare une salle le samedi matin.
- **Administrateurs** : **Droits complets** : gestion des badges, rôles, et historiques.
Exemple : Désactiver un badge perdu ou volé.

Parties prenantes :

- **Étudiants/Staff** : Utilisateurs quotidiens -> besoin de **fluidité**.
 - **Invités** : Accès ponctuel -> besoin de **simplicité**.
 - **Administrateurs** : Décideurs -> priorité à la **sécurité**.
- Enjeu commun* : **Équilibrer, accessibilité** (pour les utilisateurs) et **contrôle** (pour l'école).

La cartographie des parties prenantes facilite la communication, la priorisation des besoins et la gestion des attentes tout au long du projet. Cette approche systémique permet de concevoir une solution alignée à la fois sur les usages réels et les contraintes organisationnelles.

User Stories prioritaires et critères d'acceptation :

User Stories prioritaires :

- En tant qu'**Étudiant ou membre du staff**, je souhaite accéder au bâtiment afin de pouvoir travailler dans de bonnes conditions.
- En tant qu'**Invité**, je souhaite accéder au bâtiment afin d'y intervenir pendant la durée autorisée.
- En tant qu'**Administrateur**, je souhaite accéder au bâtiment, consulter l'historique des entrées/sorties et gérer les rôles et badges des utilisateurs.

Critères d'acceptation :

- Étant donné que je suis un utilisateur autorisé (Étudiant, staff, Invité ou Administrateur) et que je possède un badge valide, lorsque je présente mon badge au lecteur, alors le système vérifie sa validité en base de données et autorise l'ouverture de la porte.
- Étant donné que je suis administrateur et que je me trouve sur la page de connexion, lorsque je saisis un identifiant et un mot de passe valides et que je clique sur le bouton *Connexion*, alors le système m'authentifie et m'accorde l'accès à l'application de gestion.

2.4 Exigences fonctionnelles

Les exigences fonctionnelles décrivent de manière précise les comportements attendus du système afin de répondre aux besoins métier identifiés. Elles couvrent les fonctionnalités côté utilisateur (Front Office), les fonctionnalités administratives (Back Office), la gestion des rôles et des droits d'accès, ainsi que les mécanismes de sécurité, de confidentialité et d'authentification. Ces exigences constituent une référence essentielle pour le développement, les tests d'acceptation et la validation du projet.

La sécurité et la confidentialité des données représentent des exigences critiques qui influencent directement les choix d'architecture et les décisions techniques. Une authentification robuste et une gestion fine des autorisations sont indispensables pour garantir la protection des données sensibles.

2.4.1 Fonctionnalités « Front Office »

Voici les fonctionnalités **Front office** autrement dit les

Fonctionnalités utilisateurs :

Les étudiants, membres du staff et invités accèdent uniquement à leur badge, qu'il soit physique ou dématérialisé via un smartphone.

- **Badge d'accès** : Présentation du badge permettant l'accès au bâtiment selon les droits attribués.

2.4.2 Fonctionnalités « Back Office »

Voici les fonctionnalités **Front office** autrement dit les

Fonctionnalités administrateur :

- Gérer les badges (création, suspension, réactivation).
- Attribuer des **rôles** (étudiant, staff, invité).
- Exporter les **rapports d'activité** (ex. : fréquentation par jour).

2.4.3 Confidentialité et RGPD

Protection des données :

- **Chiffrement** : Données personnelles protégées en base.
- **Conformité RGPD** : Droit d'accès, suppression, et portabilité des données.
- **Journalisation** : Traçabilité des actions sensibles (ex. : modification d'un badge).

2.4.4 Droits d'accès

Cette sous-section définit la matrice des permissions selon les rôles utilisateurs.

Permissions par rôle :

Rôle	Accès admin	Accès badge
Administrateur	✓	✓
Coach/Staff	x	✓
Étudiant	x	✓
Invité	x	✓*

*Badges temporaires (durée limitée)

2.4.5 Authentification

Cette sous-section décrit le mécanisme d'authentification et de gestion des sessions utilisateurs.

Systeme d'authentification :

- **Connexion** : Authentification par identifiant et mot de passe avec génération de jetons depuis une socket LiveView.
- **Sécurité** : Politique de mots de passe forts et verification des informations envoyer
- **Récupération de compte** : Procédure de réinitialisation du mot de passe par email sécurisé.

Les attributions de socket sont des valeurs avec état conservées côté serveur dans Phoenix.LiveView.Socket. Cela diffère du modèle HTTP sans état courant qui consiste à envoyer l'état de la connexion au client sous la forme d'un jeton ou d'un cookie et à reconstruire l'état sur le serveur pour traiter chaque requête.

2.5 Exigences et choix techniques

L'architecture en **trois tiers** (présentation, logique métier, données) permet une séparation claire des responsabilités et favorise la maintenabilité, l'évolutivité et la testabilité du système. PostgreSQL est utilisé pour garantir la cohérence transactionnelle et l'intégrité des données métier critiques.

2.5.1 Exigences non fonctionnelles

Elle couvre notamment les aspects de performance, de sécurité, de compatibilité multiplateforme et de maintenabilité.

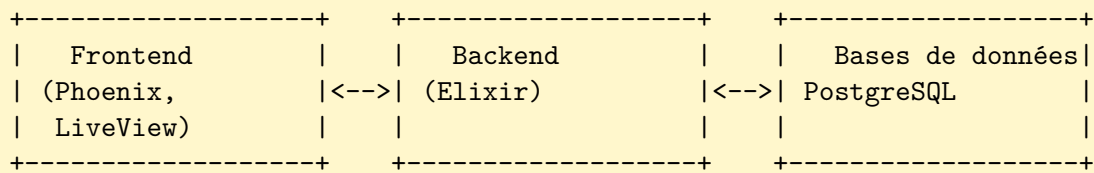
Exigences techniques :

- **Performance** : Temps de réponse inférieur à 5 secondes pour les opérations critiques.
- **Sécurité** : Chiffrement des données sensibles, journalisation des accès et mise en place d'un digicode en complément du badge sur des plages horaires définies.
- **Multiplateforme** : Compatibilité avec les smartphones iOS et Android, ainsi qu'avec de potentiels badges physiques.
- **Maintenabilité** : Code documenté, structuré et couvert par des tests automatisés.

2.5.2 Choix technologiques

Pourquoi Elixir et Phoenix ?

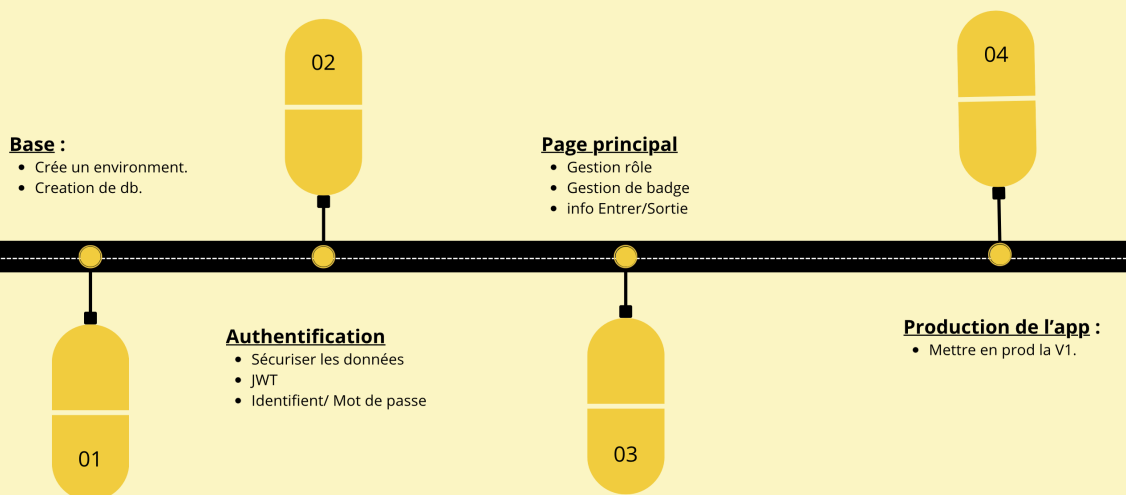
- **Elixir** : Intégration native avec l'intranet existant de Colint (déjà en Elixir). -> *Cohérence logicielle et maintenance simplifiée.*
- **Phoenix/LiveView** :
 - Interfaces réactives **sans JavaScript lourd**.
 - Logique centralisée côté serveur (moins de vulnérabilités).
- **PostgreSQL** : Base relationnelle pour les **transactions critiques** (ex. : historique des accès).

Architecture logique simplifiée :**Exemple de modèle PostgreSQL :**

```

1 CREATE TABLE users (
2     id SERIAL PRIMARY KEY,
3     email VARCHAR(255) UNIQUE NOT NULL,
4     role_id INTEGER REFERENCES roles(id),
5     badge_id INTEGER
6 );
7
8 CREATE INDEX idx_users_email ON users(email);

```

Diagramme de périmètre V1 :**Roadmap for V1**

2.6 Roadmap

La roadmap décrit l'évolution du projet, avec des ajouts **itératifs** basés sur les retours utilisateurs.

Ma roadmap :**Roadmap v1 ➡ v2 :**

- **v1.0** : Diagramme de périmètre ci-dessus.
- **v1.1** : Compatibilité complète avec les smartphones (badge dématérialisé).
- **v1.2** : Intégration de services externes via des API.
- **v2.0** : Intégration complète au système intranet de Colint.

2.7 Liens utiles

- User Stories : <https://www.mountaingoatsoftware.com/agile/user-stories>
- Priorisation MoSCoW : <https://www.productplan.com/glossary/moscow-prioritization/>
- Documentation PostgreSQL : <https://www.postgresql.org/docs/>
- Modélisation MongoDB : <https://bit.ly/mongodb-modeling>
- Architecture 3-tiers : https://en.wikipedia.org/wiki/Multitier_architecture

Chapitre 3

Méthodologie et organisation

3.1 Gestion de projet avec GitHub

Cette section présente l'approche de gestion de projet adoptée, entièrement centralisée autour de l'écosystème GitHub. L'objectif est d'assurer une planification claire, un suivi précis de l'avancement et une traçabilité complète des développements, de la conception jusqu'à la livraison.

Les outils GitHub sont utilisés à la fois comme support de collaboration, de gestion des tâches et de contrôle qualité, garantissant une organisation structurée et évolutive du projet.

Mon utilisation de GitHub :

- **Dépôt central** : Code source et documentation versionnés.
- **GitHub Projects** : Tableau Kanban pour organiser les tâches.
- **GitHub Actions** : Automatisation des tests et de l'intégration continue.

Exemple : J'utilise les **issues** pour suivre les feature et les **milestones** pour marquer les étapes clés (ex. : « Livraison MVP »).

3.1.1 User Stories et estimation de temps

Chaque **User Story** est liée à des **commits**, des **issues** et des **milestones** GitHub. Cela me permet de :

- Suivre l'avancement des fonctionnalités.
- Garantir la traçabilité des développements.

Mon tableau Kanban :

Lien vers le projet :

<https://github.com/users/Theoden-bernard/projects/2/views/2>

Colonnes et règles :

- **Backlog** : Idées et fonctionnalités non planifiées.
- **Ready** : Tâches prêtes pour le sprint suivant.
- **In Progress** : Développement en cours (**max 3 tâches simultanées**).
- **In Review** : Code en attente de validation.
- **Done** : Fonctionnalités livrées et testées.

Exemple : Une User Story comme « Créer le système d'authentification » passe de **Backlog** à **Done** en 1 semaine, avec des commits liés (ex. : FEAT: add login system).

3.2 Versioning GitHub et conventions

J'ai adopté le modèle **Git Flow** pour structurer le développement :

- Séparation claire entre le code stable (**main**), le développement (**develop**), les features (**feature**) et les correctifs (**hotfix**).
- Utilisation systématique des **Pull Requests** pour les revues de code.

Normes GitHub

Schéma Git Flow :

```
main -----  
|  
+-- develop -----  
|  
+-- feature/user-authentication  
+-- feature/project-management  
+-- hotfix/critical-bug-fix
```

Conventions de commit :

```
1 FEAT: add user authentication system (#20)  
2 FIX: resolve login validation issue (#20)  
3 DOCS: update documentation (#4)  
4 TEST: add unit tests for user service (#10)  
5 REFACTOR: improve code structure (#12)
```

Pull Requests :

- Toute modification passe par une PR.
- Revue obligatoire avant fusion dans **develop**.
- Description claire des changements (ex. : *« Ajout de la validation des mots de passe »*).

Pourquoi ?

Même seul sur le projet, ces conventions m'aident à garder un historique propre et à isoler les fonctionnalités en développement.

3.3 Planification et outils de suivi

J'ai combiné deux outils GitHub pour organiser le projet :

- **Roadmap** : Vue macro des phases clés (ex. : « Conception », « Développement MVP »).
- **Projects** : Pilotage quotidien via un tableau Kanban.

Ma planification :**Roadmap (exemple) :**

Phase 1: Conception (4 semaines)
 +-- Analyse des besoins (1 semaine)
 +-- Conception technique (2 semaines)
 +-- Validation architecture (1 semaine)

Phase 2: MVP (8 semaines)
 +-- Base de données (L)
 +-- Authentification (S)
 +-- Frontend Phoenix (M)

Configuration de GitHub Projects :

- **Colonnes** : Backlog → To Do → In Progress → Review → Done.
- **Limites WIP** : 3 tâches max par développeur (pour éviter la surcharge).
- **Automatisation** : Mise à jour des statuts en fonction des commits (ex. : un FIX passe automatiquement en **In Review**).

Bilan :

Cette organisation me permet de :

- Conserver une vision globale (roadmap).
- Réagir rapidement aux imprévus (Kanban).

3.4 Estimation de charge et planification

J'ai estimé la charge de travail pour chaque fonctionnalité afin de valider la faisabilité du projet et de prioriser les tâches critiques (ex. : authentification avant les tests).

Exemple d'estimation détaillée :

Fonctionnalité	Description	Unité	Qté	Charge unitaire	Charge totale
Authentification	Login / Register	jours	3	2	6
Gestion des projets	CRUD projets	jours	5	3	15
Tableaux de bord	Dashboard	jours	2	4	8
Tests	Tests unitaires	jours	10	1	10
Déploiement	CI/CD	jours	3	2	6
				Charge totale	45 jours

Lien vers la roadmap :

<https://github.com/users/Theoden-bernard/projects/2/views/4>

Chapitre 4

Conception fonctionnelle et technique

Ce chapitre présente l'ensemble de la conception fonctionnelle et technique du projet. Cette phase constitue un pilier fondamental de la réussite du système, car elle permet d'anticiper les contraintes, de structurer les choix techniques et de sécuriser le développement avant toute implémentation.

La conception a été menée de manière rigoureuse et progressive afin de garantir la cohérence globale du système, la maintenabilité du code et l'adéquation avec les besoins métier identifiés.

Pourquoi la phase de conception est déterminante

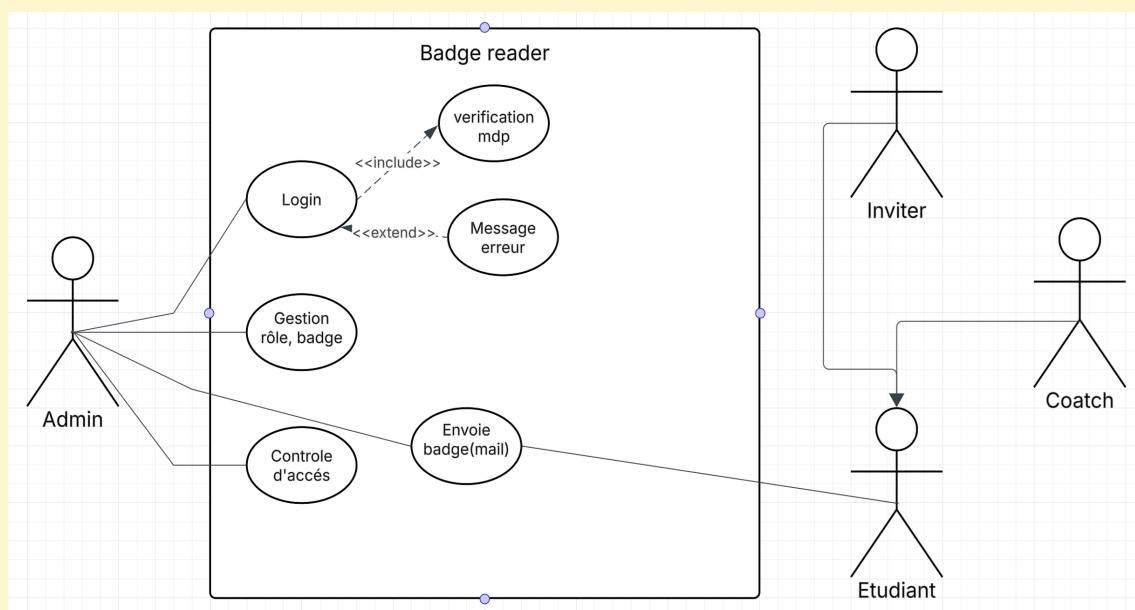
- **Réduction des coûts de refactorisation** : Une architecture bien pensée limite les reprises de code
- **Pilotage du développement** : Chaque fonctionnalité est clairement définie avant implémentation
- **Facilitation des tests** : Les cas d'usage servent de base aux scénarios de test
- **Maîtrise des risques** : Les problèmes techniques et fonctionnels sont identifiés en amont
- **Amélioration de la communication** : Une vision commune est partagée entre tous les acteurs

Contenu du chapitre :

- ✓Diagrammes de cas d'usage (Use Cases) validés
- ✓Diagrammes de séquence pour les flux critiques
- ✓Modélisation des données (MCD, MLD, MPD)
- ✓Architecture technique justifiée
- ✓User stories avec critères d'acceptation
- ✓Wireframes et maquettes validés
- ✓Charte graphique cohérente
- ✓Stratégie de tests définie

4.1 Cas d'usage et modélisation UML

Les cas d'usage modélisent les interactions entre les acteurs et le système afin d'identifier précisément les fonctionnalités attendues. Cette approche centrée utilisateur garantit une adéquation directe entre les besoins métier et les fonctionnalités implémentées. Les diagrammes UML constituent un support de communication essentiel entre les parties prenantes techniques et fonctionnelles, réduisant les risques d'ambiguïté et d'interprétation.

Diagramme de cas d'usage simplifié :**4.2 Diagrammes de séquence**

Les diagrammes de séquence décrivent les interactions temporelles entre les composants du système pour chaque cas d'usage critique. Ils permettent d'identifier clairement les responsabilités de chaque couche de l'architecture 3 tiers. Cette modélisation facilite l'implémentation des flux, la gestion des erreurs et la conception des tests d'intégration.

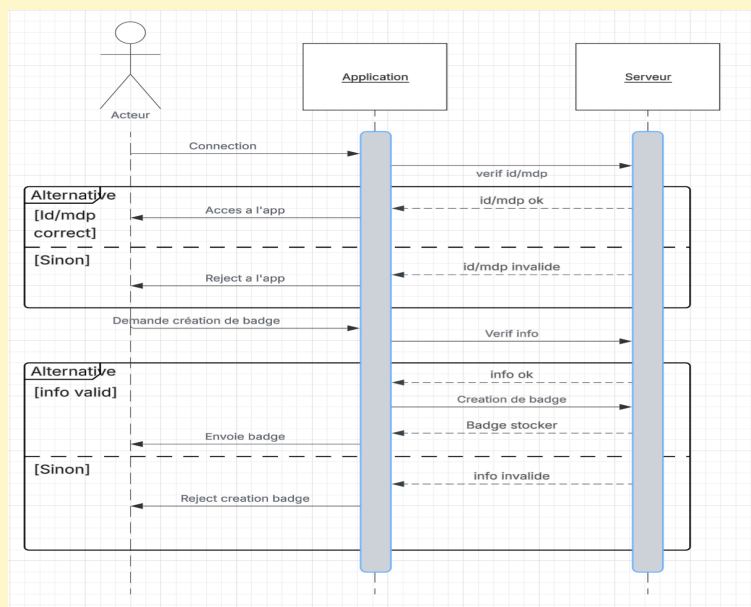
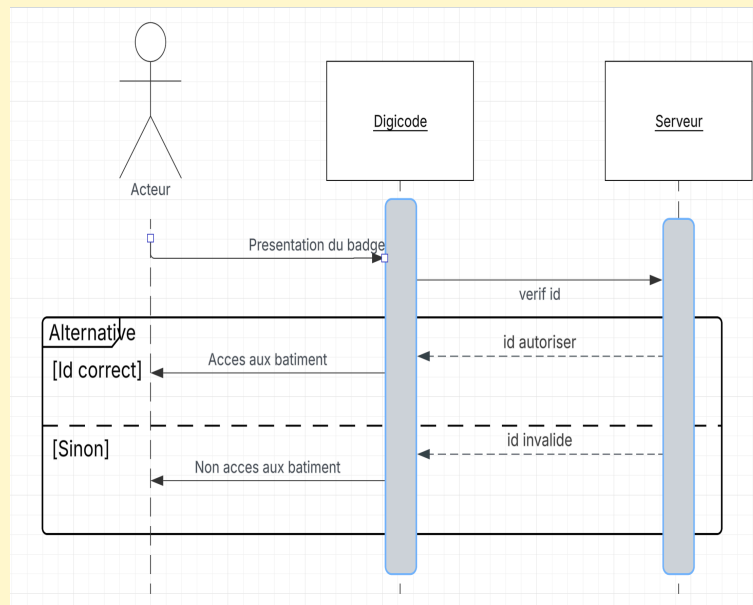
Diagramme de séquence accès site :

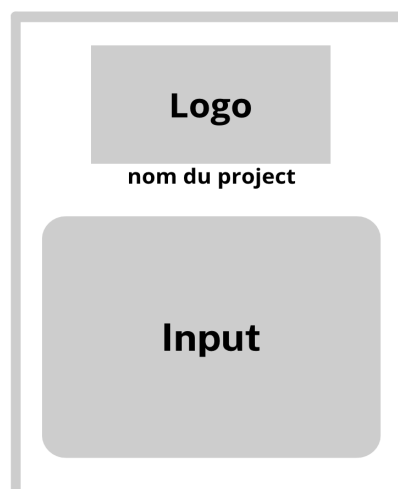
Diagramme de séquence accès bâtiment :

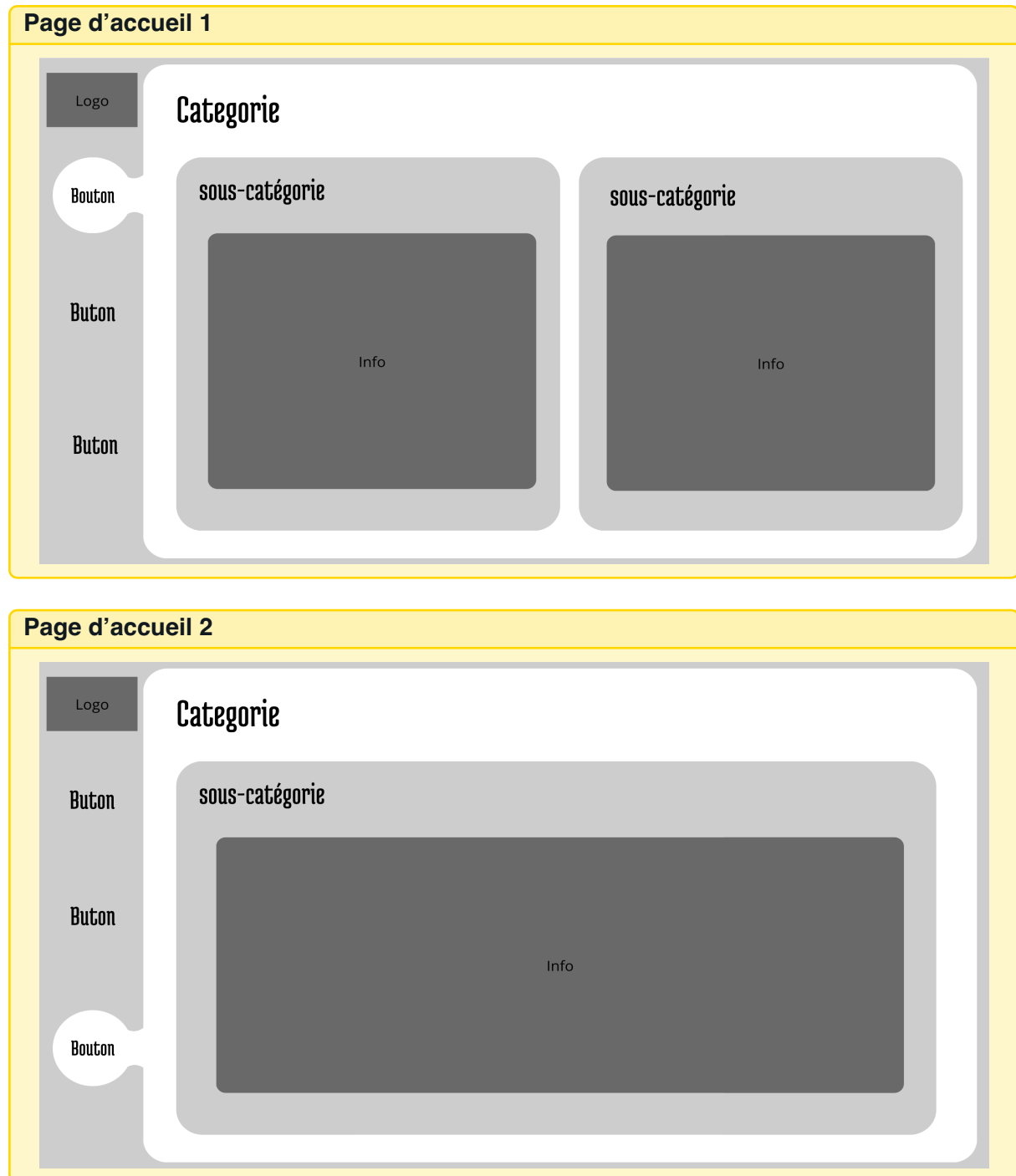
4.3 Conception de l'interface utilisateur

La conception de l'interface repose sur une démarche progressive : zoning, wireframes puis maquettes haute fidélité. Cette méthode permet de valider l'expérience utilisateur avant toute implémentation technique. L'UX a été pensée pour favoriser la simplicité, la lisibilité et l'efficacité, afin de réduire la courbe d'apprentissage et d'optimiser l'adoption du système.

4.3.1 Zoning

Dans cette sous-section, je vais vous présenter l'organisation spatiale de mes interfaces. Nous verrons une analyse claire de la hiérarchie visuelle et de l'organisation des éléments.

Page de connexion



4.3.2 Wireframe

Dans cette sous-section, je vais vous présenter mes wireframes pour les pages principales. Nous verrons une représentation claire de la structure et des interactions.

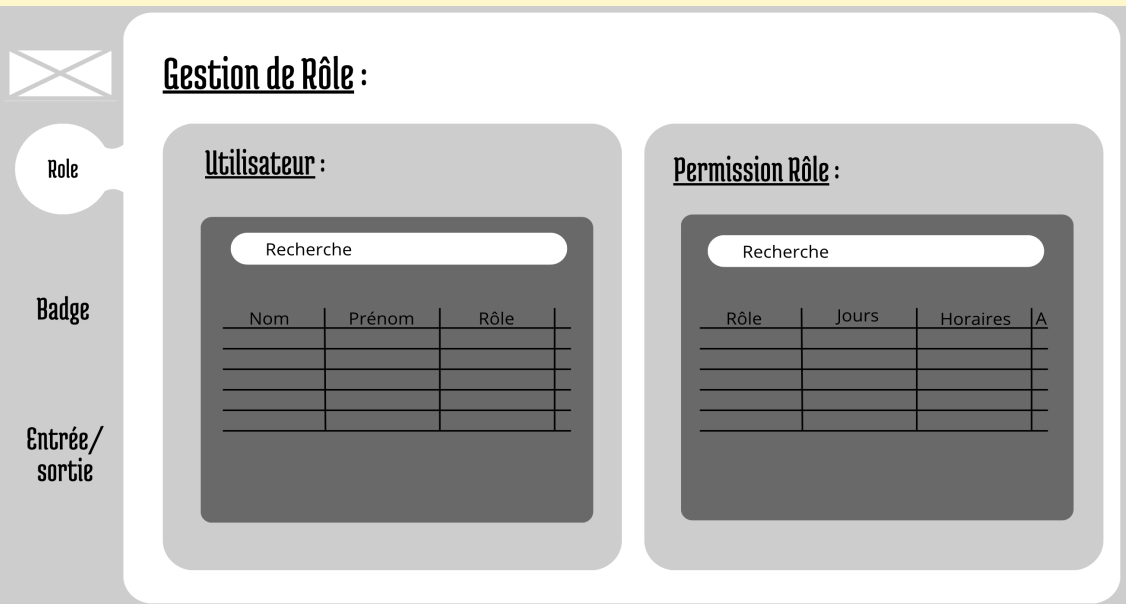
Page de connexion

Badge reader

Connection :

Identifiant

Mot de passe

Page d'accueil (Rôle)

Gestion de Rôle :

Utilisateur :

Recherche

Nom	Prénom	Rôle

Permission Rôle :

Recherche

Rôle	Jours	Horaires	A

Role

Badge

Entrée/sortie

Page d'accueil (badge)

Maquette de la page d'accueil (badge). Le design comprend une barre latérale grise à gauche avec un menu déroulant (icône d'engrenage) et trois boutons : "Rôle", "Badge" (surligné), et "Entrée/sortie". Le contenu principal est divisé en deux sections :

Gestion de Badge :

Utilisateur :

Nom	Prénom	Badge

Badge :

Badge	User	Status	A

Page d'accueil (contrôle)

Maquette de la page d'accueil (contrôle). Le design est similaire à la page précédente, avec la même barre latérale grise. Le contenu principal est :

Controle :

Entrée/sortie :

Nom	Prénom	Badge	Heure

4.3.3 Maquettage

Dans cette sous-section, je vais vous présenter mes maquettes. Nous verrons une représentation visuelle fidèle au rendu final.

Page de connexion

Colint School
Badge reader

Connection :

[Mot de passe oublier](#)

Page d'accueil (Rôle)

Colint School
Badge reader

Role

Badge

Entrée/sortie

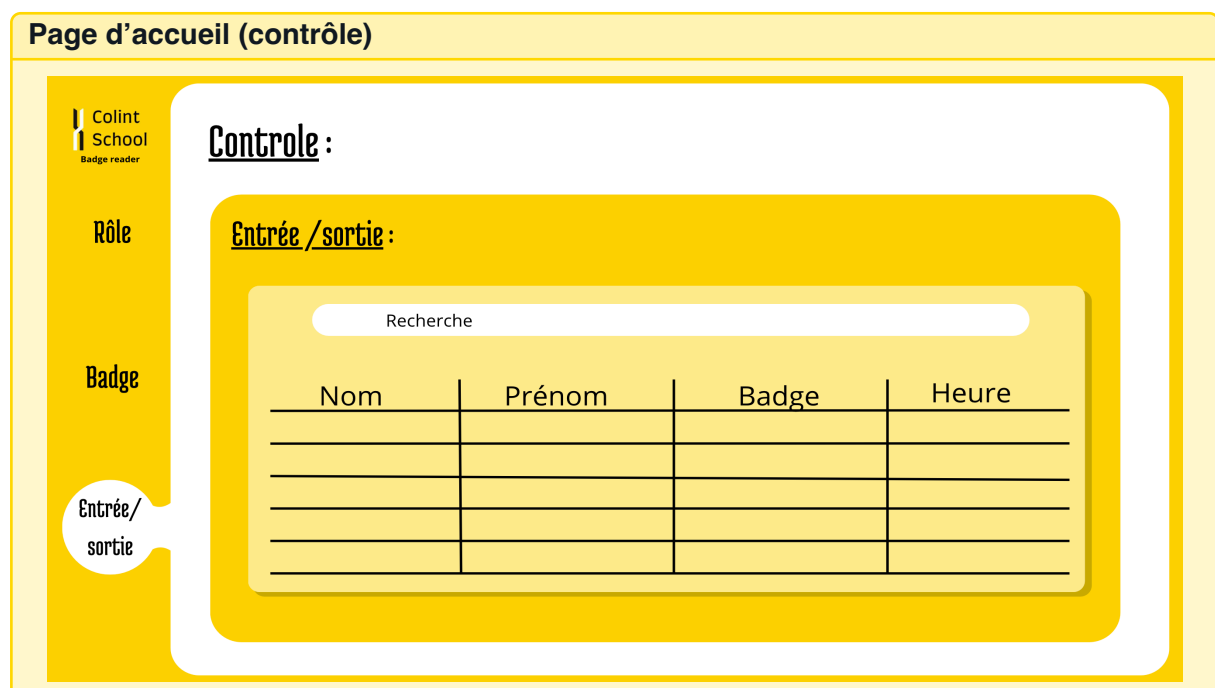
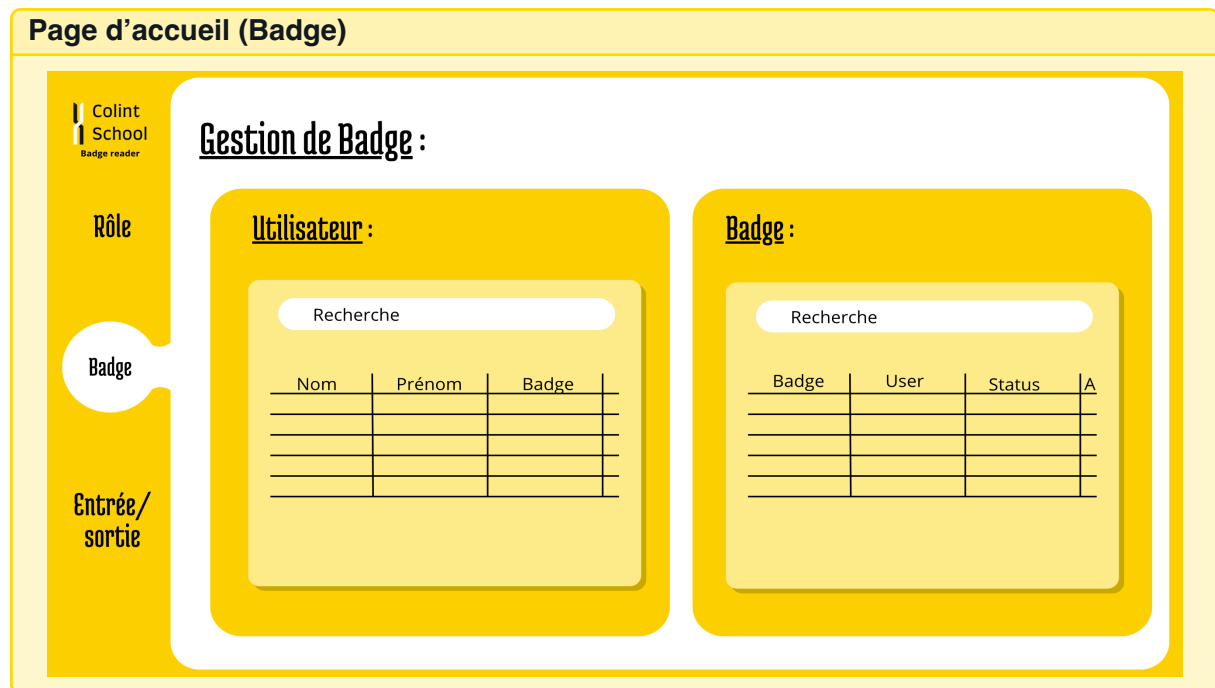
Gestion de Rôle :

Utilisateur :

Nom	Prénom	Rôle

Permission Rôle :

Rôle	Jours	Horaires	A



4.3.4 Outils de conception et diagrammes

Dans cette sous-section, je vais vous présenter les outils que j'ai utilisés pour créer mes diagrammes. Nous verrons des diagrammes clairs et bien conçus qui facilitent la compréhension de votre architecture.

Mes outils de diagrammes : *J'utilise Lucidchart ; c'est gratuit, sur internet et pratique car il dispose de beaucoup de vidéos à son sujet. De plus, il a une interface moderne qui donne envie de travailler.*

4.3.5 Charte graphique

La charte graphique garantit une cohérence visuelle sur l'ensemble des interfaces. Elle s'appuie sur l'identité de l'établissement Colint afin d'assurer une intégration future naturelle au sein de l'intranet existant.

Palette de couleurs :

- Jaune : HEX : [FFD401]
- Noir : HEX : [000000]
- Blanc : HEX : [FFFFFF]

Système typographique :

J'utiliserai la police d'écriture Farro

Logo et son utilisation :

Mon logo sera celui de Colint étant donné qu'il sera utilisé pour cette école et qu'il sera, dans le futur, intégré à l'intra de Colint. Il n'y aura pas de variante de ce logo.

Charte graphique :

Couleurs	Typographie	Composants
Primaire : #FFD401	Farro (titres)	Boutons arrondis
Secondaire : #000000	Farro (corps)	Cartes avec ombres
Neutre : #FFFFFF	Monospace (code)	Icônes Material Design
Espacement	Grille 8px	Marges cohérentes

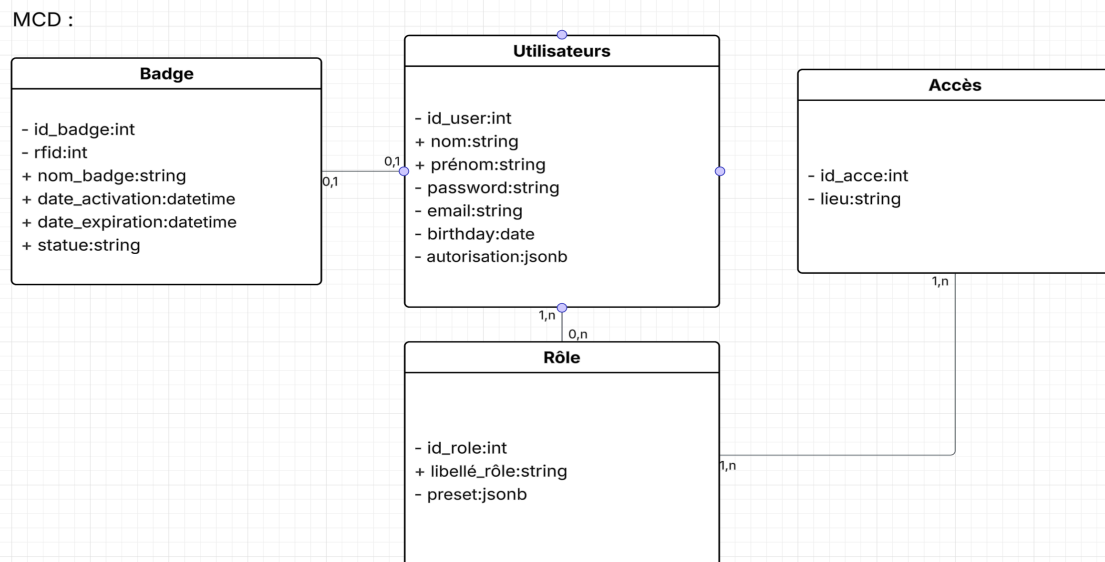
4.4 Conception de la base de données

La conception de la base de données suit la méthode Merise avec une progression structurée : Modèle Conceptuel de Données (MCD), Modèle Logique de Données (MLD) et Modèle Physique de Données (MPD). Cette approche garantit l'intégrité des données, la cohérence des relations et l'optimisation des performances. PostgreSQL est utilisé pour sa robustesse, sa gestion stricte des contraintes et sa fiabilité transactionnelle.

4.4.1 MCD (Modèle Conceptuel de Données)

Dans cette sous-section, je vais vous présenter mon modèle conceptuel de données. Nous verrons une représentation claire des entités et de leurs relations.

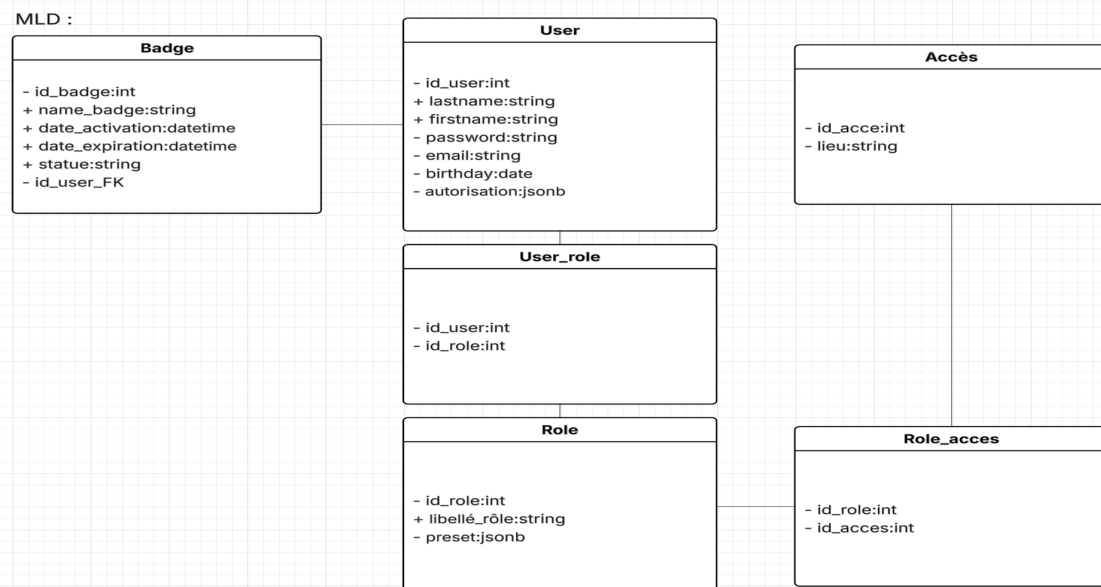
Mon MCD :



4.4.2 MLD (Modèle Logique de Données)

Dans cette sous-section, je vais vous détailler mon modèle logique de données. Nous verrons une traduction du MCD en structure de base de données.

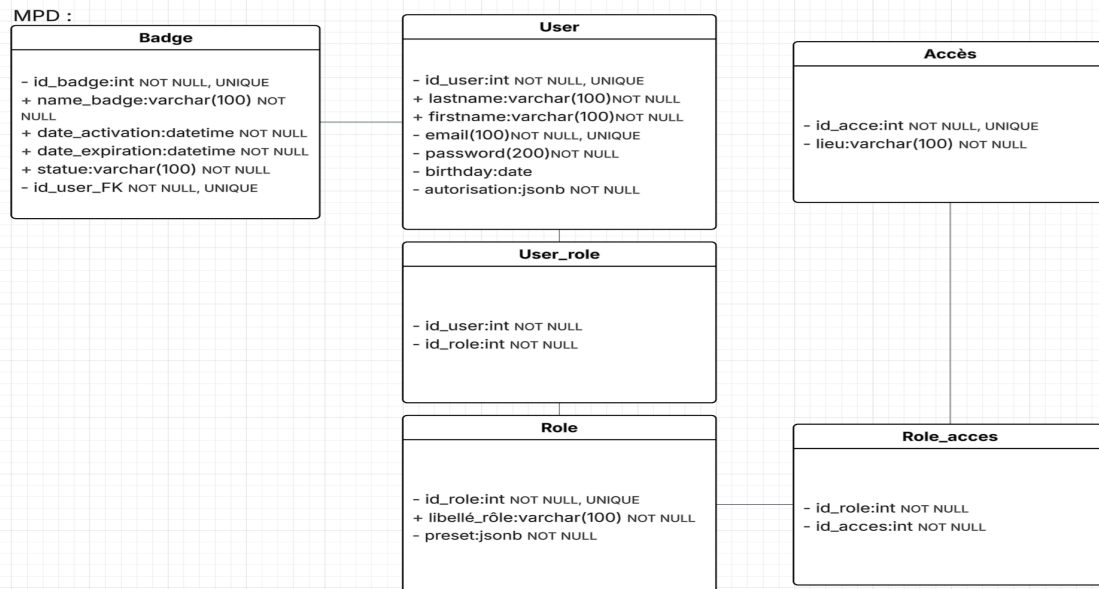
Mon MLD :



4.4.3 MPD (Modèle Physique de Données)

Dans cette sous-section, je vais vous présenter mon modèle physique de données. Nous verrons une implémentation concrète avec les contraintes et index.

Mon MPD :



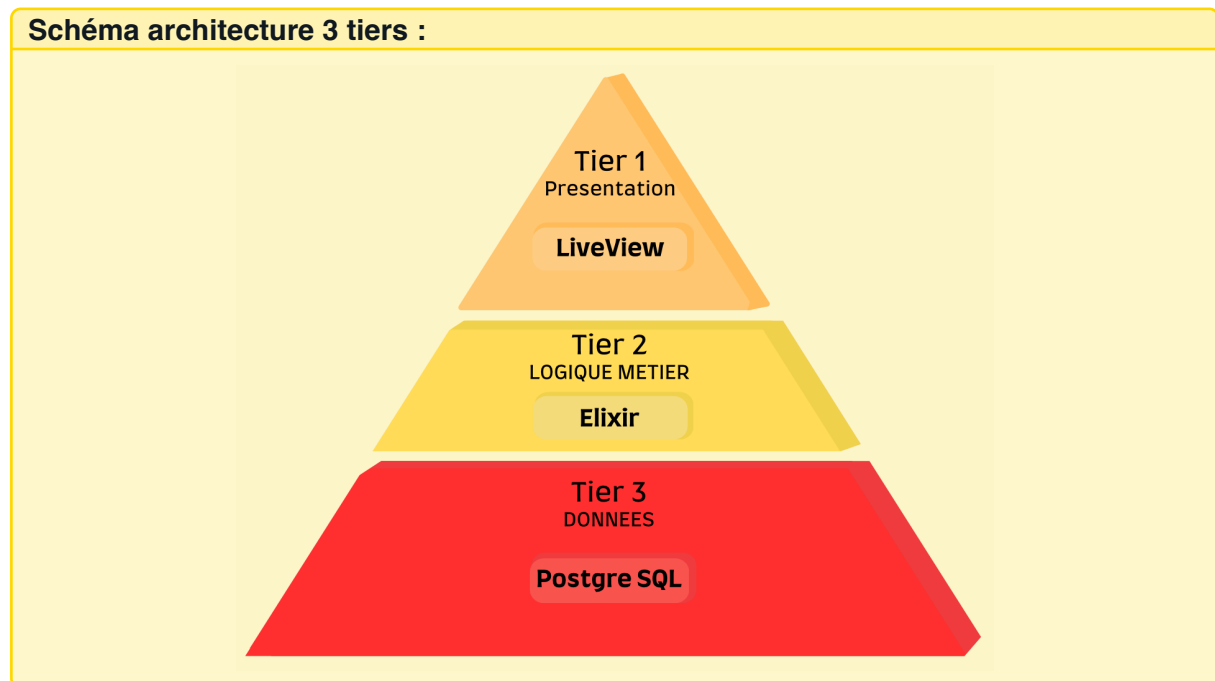
Exemple de contraintes PostgreSQL :

```

1  -- Contraintes d'intégrité
2  ALTER TABLE taches ADD CONSTRAINT fk_tache_projet
3      FOREIGN KEY (projet_id) REFERENCES projets(id)
4      ON DELETE CASCADE;
5
6  -- Index pour optimiser les performances
7  CREATE INDEX idx_taches_statut ON taches(statut);
8  CREATE INDEX idx_taches_projet_statut ON taches(projet_id, statut);
9
10 -- Contrainte de validation
11 ALTER TABLE projets ADD CONSTRAINT chk_dates
12     CHECK (date_fin > date_debut);
  
```

4.5 Architecture 3 tiers

L'architecture retenue repose sur un modèle 3 tiers, séparant clairement la couche présentation (Phoenix LiveView), la logique métier (Elixir) et la couche données (PostgreSQL). Cette séparation favorise la maintenabilité, la scalabilité et la testabilité du système. Chaque couche peut évoluer indépendamment sans impacter l'ensemble de l'architecture.



4.6 Liens utiles

- UML : <https://www.uml-diagrams.org/>
- Merise (FR) : <https://perso.liris.cnrs.fr/pierre-antoine.champin/enseignement/intro-merise.html>
- OWASP ASVS : <https://owasp.org/ASVS/>
- PostgreSQL Docs : <https://www.postgresql.org/docs/>
- MongoDB Modeling : <https://bit.ly/mongodb-modeling>
- Draw.io (diagrammes) : <https://app.diagrams.net/>
- Lighthouse (accessibilité) : <https://developers.google.com/web/tools/lighthouse>
- Lucidchart (UML) : <https://www.lucidchart.com/pages/fr/exemples/diagramme-uml>
- PlantUML (diagrammes) : <https://plantuml.com/>
- Mermaid (diagrammes) : <https://mermaid-js.github.io/mermaid/>

Chapitre 5

Architecture 3 tiers

5.1 Architecture 3 tiers

Mon architecture 3 tiers : *Cette architecture est conçue pour des applications web modernes, orientées vers la haute performance, la concurrence et l'interactivité en temps réel. Elle s'appuie sur la pile Elixir/Phoenix*

5.1.1 Couche Présentation (Frontend)

Technologies de présentation et Structure des composants :

Technologies de présentation :

- **Framework** : Phoenix 1.8.0
- **État(Pannes)** : GenServer, Agents et ETS(Erlang Term Storage)
- **Db** : Ecto
- **Routing et HTTP** : Pipeline Plug
- **UI** : liveVue 1.1

Structure des composants :

```
badge_reader/  
+-- _build/                #Compile directory  
+-- assets/  
+-- config/  
+-- deps/  
+-- lib/                   #Code directory  
|   +-- badge_reader/      #Backend directory  
|   +-- badge_reader_web/  #Frontend directory  
|   +-- badge_reader_web.ex  
|   +-- badge_reader.ex  
+-- priv/  
+-- test/                  #Test directory  
+-- .formatter.exs  
+-- AGENTS.md  
+-- mix.exs  
+-- mix.lock
```

5.1.2 Couche Logique Métier (Backend)

Couche logique métier :

Elixir utilise un environnement "mix" c'est un outil de construction "build tool" et un gestionnaire de projets qui est intégré à Elixir de manière native. il peut être comparé à "npm" de Node.js, ou de "Maven/Gradle" de Java. build est tout ce qui concerne la compilation, le code quand à lui est rangé dans lib et test comme son nom l'indique sera utilisé pour tester notre code

Controller

Vos contrôleurs : *[Décrivez vos contrôleurs et leur rôle]*

Exemple**Exemple de contrôleur :**

```

1 // ProjectController.js
2 class ProjectController {
3   async createProject(req, res) {
4     try {
5       const projectData = req.body;
6       const project = await this.projectService.create(projectData);
7       res.status(201).json(project);
8     } catch (error) {
9       res.status(400).json({ error: error.message });
10    }
11  }
12 }

```

Service

Vos services : *[Décrivez vos services et la logique métier]*

Exemple**Exemple de service :**

```

1 // ProjectService.js
2 class ProjectService {
3   async create(projectData) {
4     // Validation des données
5     this.validateProjectData(projectData);
6
7     // Logique métier
8     const project = await this.projectRepository.create(projectData);
9
10    // Notifications
11    await this.notificationService.notifyTeam(project);
12
13    return project;
14  }
15 }

```

Repository (DAO)

Vos repositories : *[Décrivez vos repositories et l'accès aux données]*

Exemple**Exemple de repository :**

```

1 // ProjectRepository.js
2 class ProjectRepository {
3   async create(projectData) {
4     const query = 'INSERT INTO projects (name, description) VALUES ($1, $2) RETURNING *';
5     const values = [projectData.name, projectData.description];
6     const result = await this.db.query(query, values);
7     return result.rows[0];
8   }
9 }

```


5.1.3 Couche Données (Database)

Dans cette sous-section, vous devez présenter la couche de données de votre application. Le jury attend une explication de l'architecture des données et des choix techniques.

Votre couche données : *[Décrivez votre architecture de données]*

Exemple

Architecture des données :

- **PostgreSQL** : Données transactionnelles et relations
- **MongoDB** : Logs, rapports et données non-structurées
- **Redis** : Cache et sessions utilisateur
- **ORM** : Prisma pour PostgreSQL, Mongoose pour MongoDB

Exemple de repository PostgreSQL :

```
1 class ProjectRepository {
2   async create(projectData) {
3     return await prisma.project.create({
4       data: {
5         name: projectData.name,
6         description: projectData.description,
7         userId: projectData.userId
8       },
9       include: {
10        tasks: true,
11        user: { select: { id: true, email: true } }
12      }
13    });
14  }
15 }
```

5.1.4 Communication entre les tiers

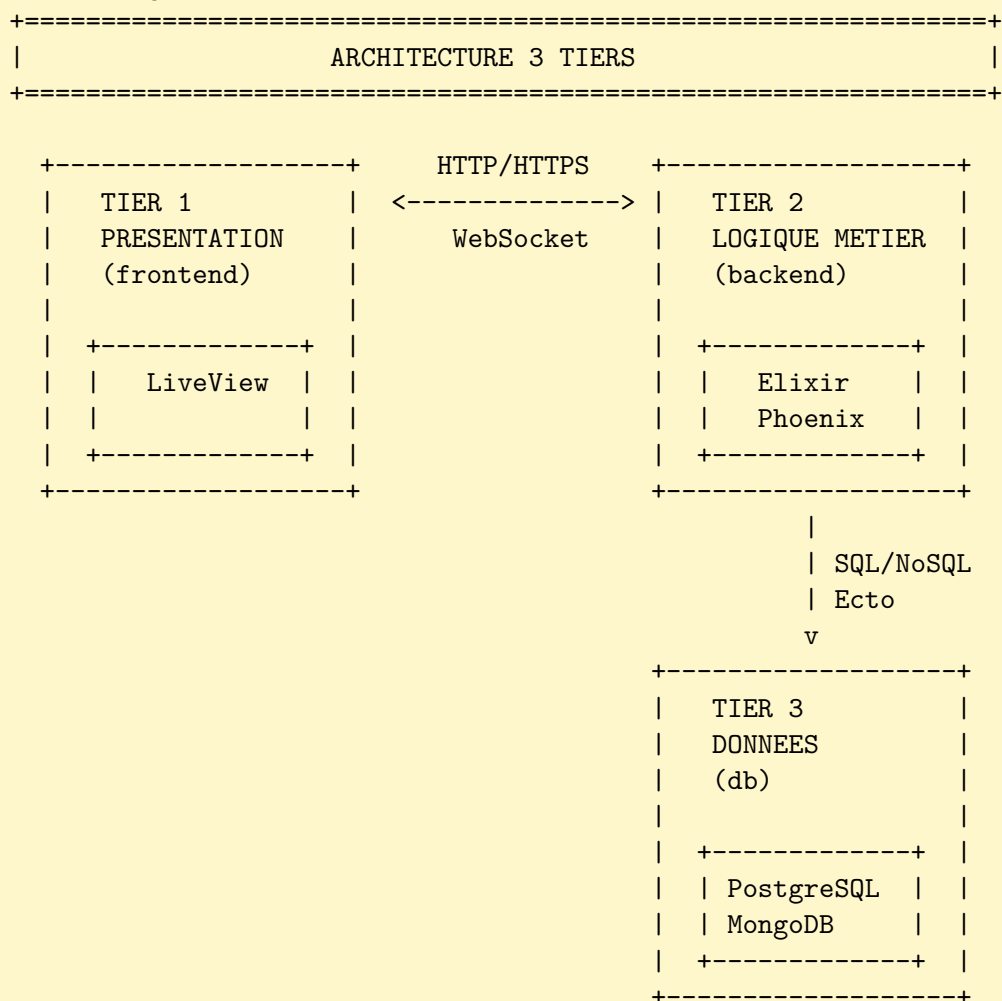
Dans cette sous-section, je vais vous expliquer comment les différents tiers communiquent entre eux. Nous verrons une compréhension claire des flux de données et des protocoles utilisés.

flux de communication :

Flux de communication 3 tiers :

L'architecture 3-Tier avec Elixir/Phoenix est particulièrement puissante car :

- **1** Le Tier 2 est le seul point d'accès au Tier 3, renforçant la sécurité.
- **2** Elixir/Phoenix (Tier 2) est l'endroit idéal pour la scalabilité horizontale grâce au BEAM, qui permet de clusteriser les serveurs d'application sans effort.
- **3** LiveView brouille légèrement la frontière entre Tier 1 et Tier 2, mais la séparation logique est maintenue : le Tier 1 s'occupe de l'affichage, et le Tier 2 (le LiveView process) gère l'état et la logique via les Contexts.

**5.1.5 Avantages de l'architecture 3 tiers**

Avantages de mon architecture : *Elixir et Phoenix sont conçu pour fonctionner en architecture 3 tiers avec des outils à disposition.*

Avantages de l'architecture 3 tiers :

- **Séparation des responsabilités** : Chaque tier a un rôle défini
- **Scalabilité** : Possibilité de scaler chaque tier indépendamment
- **Maintenabilité** : Modifications isolées par tier
- **Tests** : Tests unitaires par tier facilités
- **Sécurité** : Contrôle d'accès par tier
- **Performance** : Optimisation possible par tier

5.2 Développement Frontend

Dans cette section, je vais vous présenter mon approche de développement frontend et expliquer mes choix techniques. Nous verrons une compréhension claire de mon architecture frontend et des bonnes pratiques appliquées.

Technologies choisies : *J'ai choisi le Framework Phoenix car il met à disposition un outil qui se nomme LiveView. C'est un outil parfait pour faire du frontend car il a des caractéristiques intéressantes comme :*

- Interactions en Temps Réel.
- Sécurité Renforcée (traitement des données restent sur le serveur).
- Optimiser pour Elixir.

Architecture des composants : *Mon architecture est basée sur l'environnement mix fournie par Elixir tout mon code se trouve dans lib/ et à l'intérieur de lib se trouvent mes dossiers backend et frontend.*

Accessibilité et UX : *[Expliquez comment vous respectez les standards RGAA/WCAG et utilisez Lighthouse pour mesurer les performances]*

Structure frontend :

```
badge_reader/
+-- _build/                #Compile directory
+-- assets/
+-- config/
+-- deps/
+-- lib/                  #Code directory
|   +-- badge_reader/    #Backend directory
|   +-- badge_reader_web/ #Frontend directory
|   |   +-- components/
|   |   +-- controllers/
|   |   +-- endpoint.ex
|   |   +-- gettext.ex
|   |   +-- router.ex
|   |   +-- telemetry.ex
|   +-- badge_reader_web.ex
|   +-- badge_reader.ex
+-- priv/
+-- test/                #Test directory
+-- .formatter.exs
+-- AGENTS.md
+-- mix.exs
+-- mix.lock
```

Exemple de composant accessible :

```

1 const ProjectCard = ({ project, onEdit }) => {
2   return (
3     <div
4       className="project-card"
5       role="article"
6       aria-labelledby={`project-${project.id}-title`}
7     >
8       <h3 id={`project-${project.id}-title`}
9         {project.name}
10      </h3>
11      <p>{project.description}</p>
12      <button
13        onClick={() => onEdit(project.id)}
14        aria-label={`Modifier le projet ${project.name}`}
15      >
16        Modifier
17      </button>
18    </div>
19  );
20 };

```

Exemple**Exemple de rapport Lighthouse (1/2) :**

```

1 {
2   "categories": {
3     "performance": { "score": 0.92 },
4     "accessibility": { "score": 0.95 },
5     "best-practices": { "score": 0.88 },
6     "seo": { "score": 0.90 }
7   }
8 }

```

Exemple**Exemple de rapport Lighthouse (2/2) :**

```

1 {
2   "audits": {
3     "first-contentful-paint": { "score": 0.95 },
4     "largest-contentful-paint": { "score": 0.88 },
5     "color-contrast": { "score": 1.0 },
6     "aria-allowed-attr": { "score": 1.0 }
7   }
8 }

```

5.3 Développement Backend

Le backend implémente une API REST avec Express.js suivant le pattern Controller/Service/Repository pour séparer les responsabilités. La validation des données utilise Joi ou Zod pour garantir la cohérence des entrées. La gestion d'erreur centralisée assure des réponses API cohérentes et facilite le debugging.

L'authentification JWT sécurise les endpoints sensibles avec des middlewares de vérification. La documentation OpenAPI/Swagger facilite l'intégration frontend et la maintenance de l'API.

Structure backend :

```

badge_reader/
+-- _build/                                #Compile directory
+-- assets/
+-- config/
+-- deps/
+-- lib/                                    #Code directory
| +-- badge_reader/                        #Backend directory
| |   +-- application.ex
| |   +-- mailer.ex
| |   +-- repo.ex
| +-- badge_reader_web/                    #Frontend directory
| +-- badge_reader_web.ex
| +-- badge_reader.ex
+-- priv/
+-- test/                                  #Test directory
+-- .formatter.exs
+-- AGENTS.md
+-- mix.exs
+-- mix.lock

```

Exemple de contrôleur avec validation :

```

1  const createProject = async (req, res, next) => {
2    try {
3      // Validation des données
4      const { error, value } = projectSchema.validate(req.body);
5      if (error) {
6        return res.status(400).json({
7          error: 'Données invalides',
8          details: error.details
9        });
10     }
11
12     // Appel du service métier
13     const project = await projectService.createProject(value, req.user.id
14       );
15
16     // Log de l'action
17     await auditService.logAction('project_created', req.user.id, {
18       projectId: project.id
19     });
20
21     res.status(201).json(project);
22   } catch (error) {
23     next(error);
24   }
25 };

```

5.4 Gestion des données

La couche données utilise un ORM (Ecto) pour PostgreSQL et le driver natif pour MongoDB. Les transactions garantissent la cohérence des données critiques, tandis que les requêtes optimisées avec des index améliorent les performances. Le pattern Repository abstrait l'accès aux données et facilite les tests.

PostgreSQL gère les données transactionnelles avec des contraintes strictes, MongoDB stocke les logs et rapports avec des pipelines d'agrégation pour les analytics. Cette séparation optimise les performances selon le type d'opération.

Exemple

Exemple de repository PostgreSQL :

```

1 class ProjectRepository {
2   async create(projectData) {
3     return await prisma.project.create({
4       data: {
5         name: projectData.name,
6         description: projectData.description,
7         userId: projectData.userId
8       },
9       include: {
10        tasks: true,
11        user: { select: { id: true, email: true } }
12      }
13    });
14  }
15
16  async findByUser(userId) {
17    return await prisma.project.findMany({
18      where: { userId },
19      include: { tasks: true }
20    });
21  }
22 }

```

Exemple de pipeline MongoDB :

```

1 // Pipeline d'agrégation pour les statistiques
2 const getProjectStats = async (projectId, dateRange) => {
3   return await activityLogs.aggregate([
4     {
5       $match: {
6         'metadata.projectId': projectId,
7         timestamp: { $gte: dateRange.start, $lte: dateRange.end }
8       }
9     },
10    {
11      $group: {
12        _id: '$action',
13        count: { $sum: 1 },
14        uniqueUsers: { $addToSet: '$userId' }
15      }
16    },
17    {
18      $project: {
19        action: '$_id',
20        count: 1,
21        uniqueUsersCount: { $size: '$uniqueUsers' }
22      }
23    }
24  ]);
25 };

```

5.5 Liens utiles

- OpenAPI/Swagger : <https://swagger.io/specification/>
- WCAG : <https://www.w3.org/WAI/standards-guidelines/wcag/>
- Lighthouse : <https://developers.google.com/web/tools/lighthouse>
- PostgreSQL Tutorial : <https://www.postgresql.org/docs/current/tutorial.html>
- MongoDB Aggregation : <https://www.mongodb.com/docs/manual/aggregation/>
- Prisma Documentation : <https://www.prisma.io/docs/>

Chapitre 6

Sécurité applicative et RGPD

6.1 Protection contre les vulnérabilités OWASP

La sécurité applicative s'appuie sur les recommandations OWASP Top 10 pour protéger contre les vulnérabilités courantes. La protection XSS utilise l'échappement automatique de React et la validation côté serveur. La prévention SQL injection repose sur les requêtes paramétrées de l'ORM Prisma. La protection CSRF implémente des tokens synchronisés et la validation des origines.

Les headers de sécurité (CSP, HSTS, X-Frame-Options) renforcent la protection au niveau HTTP. La validation stricte des entrées utilisateur et la sanitisation des données réduisent les risques d'injection et de manipulation.

Exemple**Middleware de sécurité Express :**

```

1  const helmet = require('helmet');
2  const rateLimit = require('express-rate-limit');
3
4  // Configuration Helmet
5  app.use(helmet({
6    contentSecurityPolicy: {
7      directives: {
8        defaultSrc: ['self'],
9        styleSrc: ['self', 'unsafe-inline'],
10       scriptSrc: ['self']
11     }
12   });
13
14
15  // Rate limiting
16  const limiter = rateLimit({
17    windowMs: 15 * 60 * 1000, // 15 min
18    max: 100, // 100 req/IP
19    message: 'Trop de requêtes'
20  });
21  app.use('/api/', limiter);
22
23  // Validation des entrées
24  const validateInput = (schema) => {
25    return (req, res, next) => {
26      const { error } = schema.validate(req.body);
27      if (error) {
28        return res.status(400).json({
29          error: 'Données invalides',
30          details: error.details.map(d => d.message)
31        });
32      }
33      next();
34    };
35  };

```

Protection XSS côté frontend :

```

1  // React échappe automatiquement les données
2  const UserProfile = ({ user }) => {
3    return (
4      <div>
5        <h1>{user.name}</h1> { /* Échappé automatiquement */ }
6        <p>{user.bio}</p>
7        { /* Danger : éviter dangerouslySetInnerHTML */ }
8      </div>
9    );
10 };
11
12 // Validation côté client avec Zod
13 const userSchema = z.object({
14   name: z.string().min(1).max(100),
15   email: z.string().email(),
16   bio: z.string().max(500).optional()
17 });

```

À FAIRE / À VÉRIFIER

- Implémenter les protections OWASP Top 10
- Configurer les headers de sécurité avec Helmet
- Utiliser le rate limiting pour prévenir les attaques DoS
- Valider et sanitiser toutes les entrées utilisateur
- Tester la sécurité avec des outils automatisés

Contrôles Jury CDA

- Quelles vulnérabilités OWASP avez-vous adressées ?
- Comment protégez-vous contre les attaques XSS ?
- Votre protection SQL injection est-elle efficace ?
- Avez-vous configuré les headers de sécurité ?
- Comment testez-vous la sécurité de votre application ?

6.2 Authentification et autorisation

L'authentification utilise JWT avec des tokens d'accès courts (15 minutes) et des refresh tokens sécurisés. Le hachage des mots de passe utilise Argon2, plus sécurisé que bcrypt. L'autorisation implémente un système de rôles et permissions granulaire avec des middlewares de vérification.

La gestion des sessions sécurise les tokens avec des cookies HttpOnly et SameSite. La déconnexion invalide les tokens côté serveur et côté client pour garantir la sécurité.

Exemple**Configuration JWT et Argon2 (1/3) :**

```
1 const jwt = require('jsonwebtoken');
2 const argon2 = require('argon2');
3
4 // Configuration JWT
5 const JWT_SECRET = process.env.JWT_SECRET;
6 const JWT_EXPIRES_IN = '15m';
7 const REFRESH_EXPIRES_IN = '7d';
8
9 // Hachage des mots de passe avec Argon2
10 const hashPassword = async (password) => {
11   return await argon2.hash(password, {
12     type: argon2.argon2id,
13     memoryCost: 2 ** 16, // 64 MB
14     timeCost: 3,
15     parallelism: 1
16   });
17 };
```

Exemple**Configuration JWT et Argon2 (2/3) :**

```
1 // Génération des tokens
2 const generateTokens = (userId, role) => {
3   const accessToken = jwt.sign(
4     { userId, role, type: 'access' },
5     JWT_SECRET,
6     { expiresIn: JWT_EXPIRES_IN }
7   );
8
9   const refreshToken = jwt.sign(
10    { userId, type: 'refresh' },
11    JWT_SECRET,
12    { expiresIn: REFRESH_EXPIRES_IN }
13  );
14
15  return { accessToken, refreshToken };
16 };
```

Exemple**Configuration JWT et Argon2 (3/3) :**

```

1 // Middleware d'authentification
2 const authenticateToken = (req, res, next) => {
3   const authHeader = req.headers['authorization'];
4   const token = authHeader && authHeader.split(' ')[1];
5
6   if (!token) {
7     return res.status(401).json({
8       error: 'Token d\'accès requis'
9     });
10  }
11
12  jwt.verify(token, JWT_SECRET, (err, user) => {
13    if (err) {
14      return res.status(403).json({
15        error: 'Token invalide'
16      });
17    }
18    req.user = user;
19    next();
20  });
21 };

```

Système de permissions (1/2) :

```

1 // Définition des permissions
2 const PERMISSIONS = {
3   PROJECT_CREATE: 'project:create',
4   PROJECT_READ: 'project:read',
5   PROJECT_UPDATE: 'project:update',
6   PROJECT_DELETE: 'project:delete',
7   USER_MANAGE: 'user:manage'
8 };
9
10 // Rôles et leurs permissions
11 const ROLES = {
12   ADMIN: [PERMISSIONS.PROJECT_CREATE, PERMISSIONS.PROJECT_READ,
13     PERMISSIONS.PROJECT_UPDATE, PERMISSIONS.PROJECT_DELETE,
14     PERMISSIONS.USER_MANAGE],
15   MANAGER: [PERMISSIONS.PROJECT_CREATE, PERMISSIONS.PROJECT_READ,
16     PERMISSIONS.PROJECT_UPDATE],
17   DEVELOPER: [PERMISSIONS.PROJECT_READ, PERMISSIONS.PROJECT_UPDATE]
18 };

```

Système de permissions (2/2) :

```

1 // Middleware d'autorisation
2 const requirePermission = (permission) => {
3   return (req, res, next) => {
4     const userPermissions = ROLES[req.user.role] || [];
5     if (!userPermissions.includes(permission)) {
6       return res.status(403).json({
7         error: 'Permissions insuffisantes'
8       });
9     }
10    next();
11  };
12 };

```

À FAIRE / À VÉRIFIER

- Utiliser JWT avec des tokens courts et refresh tokens
- Implémenter Argon2 pour le hachage des mots de passe
- Créer un système de rôles et permissions granulaire
- Sécuriser les cookies avec HttpOnly et SameSite
- Implémenter la déconnexion sécurisée

Contrôles Jury CDA

- Pourquoi utiliser JWT plutôt que des sessions ?
- Comment gérez-vous la sécurité des mots de passe ?
- Votre système d'autorisation est-il granulaire ?
- Comment gérez-vous l'expiration des tokens ?
- Avez-vous prévu la révocation des tokens ?

6.3 Conformité RGPD

La conformité RGPD implique la mise en place d'un registre des traitements, la minimisation des données collectées, et la sécurisation des données personnelles. Le consentement explicite est recueilli pour chaque traitement, avec la possibilité de retrait. Les droits des personnes (accès, rectification, effacement, portabilité) sont implémentés via des APIs dédiées.

La protection des données utilise le chiffrement au repos et en transit, avec des sauvegardes sécurisées. La notification des violations de données est automatisée pour respecter le délai de 72h.

Exemple**Registre des traitements :**

```

1 // Modèle de registre des traitements
2 const dataProcessingRegistry = {
3   'user-authentication': {
4     purpose: 'Authentification et gestion des comptes utilisateurs',
5     legalBasis: 'Consentement',
6     dataCategories: ['identité', 'connexion'],
7     retentionPeriod: '3 ans après fermeture du compte',
8     recipients: ['équipe technique', 'hébergeur'],
9     transfers: ['UE', 'États-Unis (clauses contractuelles)']
10  },
11  'project-management': {
12    purpose: 'Gestion des projets et collaboration',
13    legalBasis: 'Exécution du contrat',
14    dataCategories: ['travail', 'communication'],
15    retentionPeriod: '5 ans après fin du projet',
16    recipients: ['équipe projet', 'clients'],
17    transfers: ['UE uniquement']
18  }
19 };
20
21 // API pour les droits RGPD
22 const gdprController = {
23   // Droit d'accès
24   async getPersonalData(req, res) {
25     const userId = req.user.userId;
26     const userData = await userService.getCompleteUserData(userId);
27     res.json({
28       personalData: userData,
29       processingPurposes: Object.keys(dataProcessingRegistry),
30       retentionPeriods: dataProcessingRegistry
31     });
32   },
33
34   // Droit à l'effacement
35   async deletePersonalData(req, res) {
36     const userId = req.user.userId;
37     await userService.anonymizeUserData(userId);
38     await auditService.logAction('gdpr_deletion', userId);
39     res.json({ message: 'Données personnelles supprimées' });
40   },
41
42   // Droit à la portabilité
43   async exportPersonalData(req, res) {
44     const userId = req.user.userId;
45     const exportData = await userService.exportUserData(userId);
46     res.attachment('mes-donnees.json');
47     res.json(exportData);
48   }
49 };

```

Chiffrement des données sensibles (1/3) :

```

1 const crypto = require('crypto');
2
3 // Configuration du chiffrement
4 const ENCRYPTION_KEY = process.env.ENCRYPTION_KEY;
5 const ALGORITHM = 'aes-256-gcm';

```

Exemple**Chiffrement des données sensibles (2/3) :**

```
1 // Fonction de chiffrement
2 const encrypt = (text) => {
3   const iv = crypto.randomBytes(16);
4   const cipher = crypto.createCipher(ALGORITHM, ENCRYPTION_KEY);
5   cipher.setAAD(Buffer.from('user-data'));
6
7   let encrypted = cipher.update(text, 'utf8', 'hex');
8   encrypted += cipher.final('hex');
9
10  const authTag = cipher.getAuthTag();
11
12  return {
13    encrypted,
14    iv: iv.toString('hex'),
15    authTag: authTag.toString('hex')
16  };
17 };
```

Exemple**Chiffrement des données sensibles (3/3) :**

```
1 // Fonction de déchiffrement
2 const decrypt = (encryptedData) => {
3   const decipher = crypto.createDecipher(ALGORITHM, ENCRYPTION_KEY);
4   decipher.setAAD(Buffer.from('user-data'));
5   decipher.setAuthTag(Buffer.from(encryptedData.authTag, 'hex'));
6
7   let decrypted = decipher.update(encryptedData.encrypted, 'hex', 'utf8')
8   ;
9   decrypted += decipher.final('utf8');
10
11  return decrypted;
12 };
```

À FAIRE / À VÉRIFIER

- Créer un registre des traitements complet
- Implémenter les droits RGPD (accès, rectification, effacement)
- Chiffrer les données sensibles au repos et en transit
- Mettre en place la notification des violations
- Documenter les mesures de sécurité et de conformité

Contrôles Jury CDA

- Avez-vous établi un registre des traitements ?
- Comment implémentez-vous les droits RGPD ?
- Vos données sont-elles chiffrées ?
- Avez-vous prévu la notification des violations ?
- Comment gérez-vous le consentement des utilisateurs ?

6.4 Liens utiles

- OWASP Top 10 : <https://owasp.org/www-project-top-ten/>
- OWASP Cheat Sheets : <https://cheatsheetseries.owasp.org/>
- CNIL RGPD : <https://www.cnil.fr/fr/rgpd-de-quoi-parle-t-on>
- Argon2 : <https://github.com/P-H-C/phc-winner-argon2>
- JWT Best Practices : <https://tools.ietf.org/html/rfc8725>

Chapitre 7

Tests et qualité logicielle

7.1 Stratégie de tests

La stratégie de tests suit la pyramide de tests avec une base solide de tests unitaires, des tests d'intégration pour valider les interactions entre composants, et des tests end-to-end pour vérifier les parcours utilisateur complets. Cette approche garantit une couverture de code élevée tout en optimisant le temps d'exécution des tests.

Les tests de performance mesurent la latence P95 et le débit de l'application sous charge. Les tests de sécurité automatisés détectent les vulnérabilités communes. La qualité du code est surveillée avec SonarQube pour maintenir un niveau de qualité constant.

Exemple**Pyramide de tests :**

```

+=====+
|          TESTS E2E          |
|    (Cypress/Playwright)    |
|    Peu nombreux, lents    |
|    Couverture globale      |
+=====+
|
|
+=====+
|    TESTS D'INTEGRATION    |
|    (Jest/Supertest)       |
|    Nombre modere          |
|    Interactions composants |
+=====+
|
|
+=====+
|    TESTS UNITAIRES        |
|    (Jest)                  |
|    Nombreux, rapides      |
|    Couverture detaillee    |
+=====+

```

Exemple**Exemple de test unitaire (1/2) :**

```

1 // Test unitaire pour le service de projet
2 describe('ProjectService', () => {
3   let projectService;
4   let mockRepository;
5
6   beforeEach(() => {
7     mockRepository = {
8       create: jest.fn(),
9       findById: jest.fn(),
10      update: jest.fn(),
11      delete: jest.fn()
12    };
13    projectService = new ProjectService(mockRepository);
14  });

```

Exemple**Exemple de test unitaire (2/2) :**

```

1   describe('createProject', () => {
2     it('should create a project with valid data', async () => {
3       // Arrange
4       const projectData = {
5         name: 'Test Project',
6         description: 'Test Description',
7         userId: 'user123'
8       };
9       const expectedProject = { id: 'proj123', ...projectData };
10      mockRepository.create.mockResolvedValue(expectedProject);

```

Badge reader

```

1 // Act
2
3 const result = await projectService.createProject(projectData);
4
5 // Assert

```

À FAIRE / À VÉRIFIER

- Implémenter la pyramide de tests (unitaires, intégration, E2E)
- Maintenir une couverture de code élevée (>80%)
- Automatiser l'exécution des tests dans la CI/CD
- Tester les cas d'erreur et les cas limites
- Documenter les stratégies de test et les conventions

Contrôles Jury CDA

- Quelle est votre stratégie de tests ?
- Quelle est votre couverture de code ?
- Comment testez-vous les cas d'erreur ?
- Vos tests sont-ils automatisés ?
- Comment mesurez-vous la qualité de vos tests ?

7.2 Tests de performance

Les tests de performance utilisent k6 pour simuler des charges réalistes et mesurer les métriques clés : latence P95, débit, et taux d'erreur. Les scénarios de test couvrent les parcours utilisateur critiques et les pics de charge prévus. L'optimisation s'appuie sur l'analyse des goulots d'étranglement identifiés.

Le monitoring en production surveille les métriques de performance en temps réel avec des alertes automatiques. Les tests de charge réguliers valident la capacité de l'application à supporter la croissance du trafic.

Exemple**Script de test de performance k6 (1/2) :**

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3 import { Rate } from 'k6/metrics';
4
5 // Métriques personnalisées
6 const errorRate = new Rate('errors');
7
8 export let options = {
9   stages: [
10     { duration: '2m', target: 10 }, // Montée en charge
11     { duration: '5m', target: 50 }, // Charge normale
12     { duration: '2m', target: 100 }, // Pic de charge
13     { duration: '5m', target: 50 }, // Retour à la normale
14     { duration: '2m', target: 0 }, // Descente
15   ],
16   thresholds: {
17     http_req_duration: ['p(95)<500'], // 95% des requêtes < 500ms
18     http_req_failed: ['rate<0.1'], // Moins de 10% d'erreurs
19     errors: ['rate<0.1']
20   }
21 };
```

Exemple**Script de test de performance k6 (2/2) :**

```

1 export default function() {
2   // Test de connexion
3   let loginResponse = http.post('http://localhost:3000/api/auth/login', {
4     email: 'test@example.com',
5     password: 'password123'
6   });
7
8   check(loginResponse, {
9     'login_status_is_200': (r) => r.status === 200,
10    'login_response_time_<_200ms': (r) => r.timings.duration < 200,
11  }) || errorRate.add(1);
12
13  if (loginResponse.status === 200) {
14    const token = loginResponse.json('token');
15
16    // Test de création de projet
17    let projectResponse = http.post('http://localhost:3000/api/projects',
18      JSON.stringify({
19        name: `Test Project ${_VU}`,
20        description: 'Performance_test_project'
21      }),
22      {
23        headers: {
24          'Authorization': `Bearer ${token}`,
25          'Content-Type': 'application/json'
26        }
27      }
28    );
29
30    check(projectResponse, {
31      'project_creation_status_is_201': (r) => r.status === 201,
32      'project_creation_time_<_300ms': (r) => r.timings.duration < 300,
33    }) || errorRate.add(1);
34  }
35
36  sleep(1);
37 }

```

Exemple**Résultats de performance :**

Métrique	Objectif	Mesuré	Statut
Latence P95	< 500ms	320ms	✓
Débit	> 100 req/s	150 req/s	✓
Taux d'erreur	< 1%	0.2%	✓
CPU	< 80%	65%	✓
Mémoire	< 2GB	1.2GB	✓

À FAIRE / À VÉRIFIER

- Définir des objectifs de performance mesurables
- Utiliser k6 pour les tests de charge automatisés
- Surveiller les métriques en production
- Optimiser les goulots d'étranglement identifiés
- Planifier des tests de performance réguliers

Contrôles Jury CDA

- Quels sont vos objectifs de performance ?
- Comment mesurez-vous les performances ?
- Avez-vous identifié des goulots d'étranglement ?
- Vos tests de charge sont-ils réalistes ?
- Comment surveillez-vous les performances en production ?

7.3 Qualité du code avec SonarQube

SonarQube analyse automatiquement la qualité du code, détecte les bugs, les vulnérabilités de sécurité, et les code smells. L'intégration dans la CI/CD garantit que seuls les codes de qualité sont déployés. Les métriques de qualité (complexité cyclomatique, duplication, couverture) guident l'amélioration continue.

Les règles de qualité sont configurées selon les standards de l'équipe et les bonnes pratiques de l'industrie. Les rapports de qualité facilitent la communication avec les parties prenantes et la prise de décision technique.

Lighthouse mesure automatiquement les performances, l'accessibilité, les bonnes pratiques et le SEO des applications web. L'intégration dans la CI/CD permet de surveiller ces métriques à chaque déploiement et d'alerter en cas de régression.

Exemple**Configuration SonarQube :**

```
1 # sonar-project.properties
2 sonar.projectKey=project-management-app
3 sonar.projectName=Project Management Application
4 sonar.projectVersion=1.0
5
6 # Sources et tests
7 sonar.sources=src
8 sonar.tests=tests
9 sonar.test.inclusions=tests/**/*.test.js
10
11 # Exclusions
12 sonar.exclusions=node_modules/**/*.dist/**/*.coverage/**
13
14 # Métriques de qualité
15 sonar.javascript.lcov.reportPaths=coverage/lcov.info
16 sonar.coverage.exclusions=tests/**/*.test.js
17
18 # Règles de qualité
19 sonar.qualitygate.wait=true
20 sonar.qualitygate.timeout=300
```

Exemple**Rapport de qualité SonarQube :**

Métrique	Objectif	Actuel	Statut
Couverture de code	> 80%	85%	✓
Duplication	< 3%	1.2%	✓
Complexité cyclomatique	< 10	7.3	✓
Maintenabilité	A	A	✓
Fiabilité	A	A	✓
Sécurité	A	A	✓

Exemple de correction de code smell :

```

1 // AVANT : Méthode trop longue
2 const processUserData = (userData) => {
3   const validatedData = validateUserData(userData);
4   const processedData = transformUserData(validatedData);
5   const enrichedData = enrichWithExternalData(processedData);
6   const formattedData = formatForDatabase(enrichedData);
7   const savedData = saveToDatabase(formattedData);
8   const auditLog = createAuditLog(savedData);
9   const notification = sendNotification(auditLog);
10  return notification;
11 };
12
13 // APRÈS : Méthodes courtes et focalisées
14 const processUserData = (userData) => {
15   const validatedData = validateUserData(userData);
16   const processedData = transformUserData(validatedData);
17   return saveUserData(processedData);
18 };
19
20 const saveUserData = (data) => {
21   const enrichedData = enrichWithExternalData(data);
22   const formattedData = formatForDatabase(enrichedData);
23   const savedData = saveToDatabase(formattedData);
24   auditUserAction(savedData);
25   return savedData;
26 };

```


Focus GitHub**Intégration SonarQube dans GitHub Actions :**

```
1 name: Quality Gate
2 on: [push, pull_request]
3
4 jobs:
5   quality:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v3
9
10      - name: Setup Node.js
11        uses: actions/setup-node@v3
12        with:
13          node-version: '18'
14
15      - name: Install dependencies
16        run: npm ci
17
18      - name: Run tests
19        run: npm test -- --coverage
20
21      - name: SonarQube Scan
22        uses: SonarSource/sonarqube-scan-action@v1
23        env:
24          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
25          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Métriques de qualité GitHub :

- **Couverture** : 85% (objectif : >80%)
- **Bugs** : 0 (objectif : 0)
- **Vulnérabilités** : 0 (objectif : 0)
- **Code smells** : 12 (objectif : <20)
- **Duplication** : 1.2% (objectif : <3%)

Métriques Lighthouse :

- **Performance** : 92/100 (objectif : >90)
- **Accessibilité** : 95/100 (objectif : >90)
- **Best Practices** : 88/100 (objectif : >85)
- **SEO** : 90/100 (objectif : >85)

À FAIRE / À VÉRIFIER

- Intégrer SonarQube dans votre pipeline CI/CD
- Intégrer Lighthouse CI pour surveiller les performances web
- Définir des seuils de qualité appropriés
- Corriger les code smells et vulnérabilités détectés
- Surveiller les métriques de qualité dans le temps
- Former l'équipe aux bonnes pratiques de qualité

Contrôles Jury CDA

- Comment mesurez-vous la qualité de votre code ?
- Quels sont vos scores Lighthouse pour les performances et l'accessibilité ?
- Vos métriques de qualité sont-elles satisfaisantes ?
- Comment gérez-vous les code smells détectés ?
- Avez-vous intégré la qualité dans votre CI/CD ?
- Comment améliorez-vous la qualité en continu ?

7.4 Liens utiles

- Jest Documentation : <https://jestjs.io/docs/getting-started>
- Cypress Testing : <https://docs.cypress.io/>
- SonarQube : <https://docs.sonarsource.com/sonarqube/latest/>
- Lighthouse CI : <https://developers.google.com/web/tools/lighthouse-ci>
- k6 Performance Testing : <https://k6.io/docs/>
- Testing Best Practices : <https://testingjavascript.com/>

Chapitre 8

Déploiement et CI/CD

8.1 Containerisation avec Docker

Dans le cadre de mon PFE j'ai décidé d'utiliser Docker. Docker permet la standardisation de l'environnement de développement et de production, garantissant la reproductibilité des déploiements.

Dans un premier temps nous avons besoin d'un Dockerfile qui nous permettra par la suite de créer une image docker. Puis cette image nous pourrons par la suite la transformer en conteneurs Docker. Le Dockerfile multi-stage optimise la taille des images en séparant les phases de build et de runtime.

Dockerfile :

```
1 # Image d'elixir
2 FROM elixir:1.18
3
4 # Installation des outils
5 RUN apt-get update \
6     && apt-get install -y nodejs npm \
7     && rm -rf /var/lib/apt/list/* \
8     && apt-get update && apt-get install -y inotify-tools
9
10 # Dossier de travail
11 WORKDIR /app
12
13 # Copie des fichiers du projet dans le dossier actuel
14 COPY . .
15
16 # Mise en mode production
17 ENV MIX_ENV="prod"
18
19 # Installation des dépendances et installation de Phoenix
20 RUN set -xe \
21     && mix local.hex --force \
22     && mix archive.install hex phx_new --force \
23     && mix deps.get \
24     && mix deps.compile
25
26 # compilation
27 RUN mix compile
28
29 RUN mix assets.deploy \
30     && mix phx.digest
31
32 # lancement d'un localhost
33 CMD ["mix", "phx.server"]
```

Nous utiliserons ensuite un Docker-compose qui orchestre les services (application, base de données) pour un environnement complet.

Docker Compose pour l'environnement complet :

```
1 services:
2   web:
3     depends_on:
4       - db
5     tty: true
6     image: theoden714/badge_reader:latest
7
8     ports:
9       - "4000:4000"
10
11    volumes:
12      - ./badge_reader:/app
13
14    environment:
15      MIX_ENV: "prod"
16      DATABASE_URL: "postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}
17        @db:5432/${POSTGRES_DB}"
18      SECRET_KEY_BASE: ${SECRET_KEY_BASE}
19      PHX_HOST: ${PHX_HOST}
20
21  db:
22    image: postgres:14
23
24    ports:
25      - "5432:5432"
26
27    restart: always
28    shm_size: 128mb
29
30    environment:
31      POSTGRES_USER: ${POSTGRES_USER}
32      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
33      POSTGRES_DB: ${POSTGRES_DB}
34
35    volumes:
36      - postgres_data:/var/lib/postgresql/data
37
38 volumes:
39   postgres_data:
```

Dans mon Docker-compose j'ai utilisé des "Secrets-keys". Ce sont des outils mis à disposition pour éviter de partager des données sensibles. Ces données sont directement stockées et gérées par Github

8.2 Pipeline CI/CD avec GitHub Actions

Le pipeline CI/CD automatise les étapes de linting, build, test, scan de sécurité et déploiement. GitHub Actions exécute ces étapes à chaque push et Pull Request, garantissant la qualité du code avant intégration. Les secrets et variables d'environnement sécurisent les informations sensibles.

Le déploiement automatique vers les environnements de staging et production suit une approche blue-green pour minimiser les risques. Les rollbacks automatiques sont déclenchés en cas de détection d'anomalies.

Workflow GitHub Actions partie CI

```

1 name: CI/CD Pipeline
2
3 on:
4   push:
5     branches: ["develop"]
6
7   pull_request:
8     branches: ["main", "develop"]
9
10 jobs:
11   elixir_ci:
12     runs-on: ubuntu-latest
13     steps:
14       - name: Clonage du repo
15         uses: actions/checkout@v4
16
17       - name: Installation d Elixir et Erlan
18         uses: erlef/setup-beam@v1
19         with:
20           elixir-version: '1.18'
21           otp-version: '27'
22
23       - name: Installation des dépendances Mix
24         working-directory: ./badge_reader
25         run: mix deps.get
26
27       - name: Compilation elixir
28         working-directory: ./badge_reader
29         run: mix compile
30
31   build_and_push_image:
32     runs-on: ubuntu-latest
33     needs: elixir_ci
34     steps:
35       - name: Clonage du repo
36         uses: actions/checkout@v4
37
38       - name: Installation docker
39         uses: docker/setup-qemu-action@v3
40
41       - name: Set up Docker Buildx
42         uses: docker/setup-buildx-action@v3
43
44       - name: Login to Docker Hub
45         uses: docker/login-action@v3
46         with:
47           username: ${ secrets.LOGIN }
48           password: ${ secrets.PASSWORD }
49
50       - name: Build & Push
51         uses: docker/build-push-action@v5
52         with:
53           context: ./badge_reader
54           push: true
55           tags: ${ secrets.LOGIN }/badge_reader:latest

```

Workflow GitHub Actions partie CD :

```

1 name: cd_badge_reader
2
3 on:
4   push:
5     branches: ["main"]
6
7 jobs:
8   production:
9     runs-on: ubuntu-latest
10
11     steps:
12
13       - name: Ecriture ssh key
14         run: |
15           echo "${{ secrets.SSH_KEY_AWS }}" > key.pem
16           chmod 400 key.pem
17
18       - name: ajoute Ec2 aux known_hosts
19         run: |
20           mkdir -p ~/.ssh
21           touch ~/.ssh/known_hosts
22           chmod 644 ~/.ssh/known_hosts
23           ssh-keyscan ${ secrets.EC2_IP } >> ~/.ssh/known_hosts
24
25       - name: Connection Ec2
26         run: |
27           ssh -i key.pem ${ secrets.EC2_USER }@${ secrets.EC2_IP
28             } << 'EOF'
29           echo "Connexion OK"
30           docker-compose down
31           docker system prune -a --volumes -f
32           docker pull theoden714/badge_reader:latest
33           docker-compose up -d
34           EOF
35
36       - name: Supression de la clef
37         run: rm -f key.pem

```

8.3 Documentation et monitoring

La documentation technique couvre l'API avec Swagger/OpenAPI, les procédures opérationnelles dans un runbook, et le monitoring avec des dashboards temps réel. Les logs structurés facilitent le debugging et l'analyse des performances. Les alertes automatiques notifient l'équipe en cas d'anomalie.

Le monitoring couvre les métriques applicatives (latence, débit, erreurs) et infrastructure (CPU, mémoire, disque). Les dashboards Grafana visualisent ces métriques pour faciliter la surveillance et l'analyse des tendances.

Exemple**Documentation API Swagger :**

```
1 openapi: 3.0.0
2 info:
3   title: Project Management API
4   version: 1.0.0
5
6 paths:
7   /projects:
8     get:
9       summary: Liste des projets
10      parameters:
11        - name: page
12          in: query
13          schema:
14            type: integer
15            default: 1
16      responses:
17        '200':
18          description: Liste des projets
19          content:
20            application/json:
21              schema:
22                type: object
23                properties:
24                  data:
25                    type: array
26                    items:
27                      $ref: '#/components/schemas/Project'
28      post:
29        summary: Créer un projet
30        requestBody:
31          required: true
32          content:
33            application/json:
34              schema:
35                $ref: '#/components/schemas/ProjectInput'
36        responses:
37          '201':
38            description: Projet créé
39
40 components:
41   schemas:
42     Project:
43       type: object
44       properties:
45         id: { type: string, format: uuid }
46         name: { type: string }
47         description: { type: string }
48         createdAt: { type: string, format: date-time }
49     ProjectInput:
50       type: object
51       required: [name]
52       properties:
53         name: { type: string, minLength: 1 }
54         description: { type: string }
```

Exemple**Runbook opérationnel (1/3) :**

```
1 # Runbook - Project Management Application
2
3 ## Procédures de démarrage
4
5 ### Démarrage de l'application
6 ```bash
7 # Environnement de développement
8 docker-compose up -d
9
10 # Environnement de production
11 docker-compose -f docker-compose.prod.yml up -d
12 ```
13
14 ### Vérification de santé
15 ```bash
16 curl -f http://localhost:3000/health
17 ```
```

Exemple**Runbook opérationnel (2/3) :**

```
1 ## Procédures de maintenance
2
3 ### Sauvegarde des données
4 ```bash
5 # PostgreSQL
6 pg_dump -h localhost -U user projectdb > backup_$(date +%Y%m%d).sql
7
8 # MongoDB
9 mongodump --host localhost:27017 --db projectlogs --out backup_mongo_$(
10     date +%Y%m%d)
11 ```
12
13 ### Mise à jour de l'application
14 ```bash
15 # Pull de la nouvelle image
16 docker-compose pull
17
18 # Redémarrage avec la nouvelle image
19 docker-compose up -d
20 ```
```


Exemple**Configuration de monitoring :**

```
1 \# docker-compose.monitoring.yml
2 version: '3.8'
3
4 services:
5   prometheus:
6     image: prom/prometheus
7     ports:
8       - "9090:9090"
9     volumes:
10      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
11     command:
12       - '--config.file=/etc/prometheus/prometheus.yml'
13       - '--storage.tsdb.path=/prometheus'
14       - '--web.console.libraries=/etc/prometheus/console_libraries'
15       - '--web.console.templates=/etc/prometheus/consoles'
16
17   grafana:
18     image: grafana/grafana
19     ports:
20       - "3001:3000"
21     environment:
22       - GF_SECURITY_ADMIN_PASSWORD=admin
23     volumes:
24       - grafana_data:/var/lib/grafana
25
26   node-exporter:
27     image: prom/node-exporter
28     ports:
29       - "9100:9100"
30     volumes:
31       - /proc:/host/proc:ro
32       - /sys:/host/sys:ro
33       - /:/rootfs:ro
34
35 volumes:
36   grafana_data:
```

À FAIRE / À VÉRIFIER

- Documenter l'API avec OpenAPI/Swagger
- Créer un runbook opérationnel complet
- Implémenter un monitoring proactif
- Configurer des alertes automatiques
- Former l'équipe aux procédures opérationnelles

Contrôles Jury CDA

- Votre API est-elle documentée ?
- Avez-vous un runbook opérationnel ?
- Comment surveillez-vous votre application ?
- Vos alertes sont-elles configurées ?
- L'équipe connaît-elle les procédures d'urgence ?

8.4 Liens utiles

- Dockerfile reference : <https://docs.docker.com/reference/dockerfile/>
- Docker Compose : <https://docs.docker.com/compose/>
- GitHub Actions : <https://docs.github.com/actions>
- Postman : <https://learning.postman.com/docs/getting-started/introduction/>
- Prometheus : <https://prometheus.io/docs/>

Chapitre 9

Veille technologique et sécurité

9.1 Veille technologique stack

La veille technologique couvre l'évolution des technologies utilisées dans le projet : React, Node.js, PostgreSQL, MongoDB, et Docker. Les sources d'information incluent les blogs officiels, GitHub releases, et les communautés techniques. Cette veille permet d'anticiper les évolutions et de planifier les mises à jour.

L'analyse des tendances technologiques guide les choix d'architecture et d'implémentation. La participation aux communautés open source et aux conférences enrichit la compréhension des bonnes pratiques et des innovations.

Exemple

Sources de veille technologique :

- **Frontend** : React Blog, Next.js Releases, TypeScript Roadmap
- **Backend** : Node.js Releases, Express.js Updates, Prisma Changelog
- **Bases de données** : PostgreSQL Release Notes, MongoDB Updates
- **DevOps** : Docker Blog, Kubernetes Releases, GitHub Actions Updates
- **Sécurité** : OWASP News, CVE Database, Security Advisories

Exemple de veille React :

React 18.2.0 (Janvier 2024)

- +-- Nouvelles fonctionnalités
 - | +-- Concurrent Features stabilisées
 - | +-- Suspense amélioré
 - | +-- Server Components en production
- +-- Performances
 - | +-- Réduction de 15% du bundle size
 - | +-- Amélioration du rendu concurrent
- +-- Migration
 - +-- Breaking changes mineurs
 - +-- Guide de migration disponible

Impact sur le projet :

- **React 18** : Migration planifiée pour Q2 2024
- **Node.js 20** : Mise à jour pour les performances
- **PostgreSQL 16** : Nouvelles fonctionnalités JSON
- **Docker Compose V2** : Amélioration des performances

À FAIRE / À VÉRIFIER

- Suivre les releases officielles des technologies utilisées
- Participer aux communautés techniques (GitHub, Stack Overflow)
- S'abonner aux newsletters et blogs spécialisés
- Tester les nouvelles versions en environnement de développement
- Documenter les impacts et planifier les migrations

Contrôles Jury CDA

- Quelles sources utilisez-vous pour votre veille ?
- Comment identifiez-vous les technologies émergentes ?
- Avez-vous planifié des mises à jour technologiques ?
- Comment évaluez-vous l'impact des nouvelles versions ?
- Votre veille influence-t-elle vos choix techniques ?

9.2 Bonnes pratiques sécurité

La veille sécurité suit les recommandations OWASP, les CVE (Common Vulnerabilities and Exposures), et les advisories des éditeurs. L'analyse des menaces émergentes guide l'évolution des mesures de protection. Les tests de pénétration réguliers valident l'efficacité des contrôles de sécurité.

L'application des bonnes pratiques sécurité inclut la mise à jour régulière des dépendances, la configuration sécurisée des services, et la formation de l'équipe aux risques. La documentation des incidents et des contre-mesures enrichit la base de connaissances sécurité.

Exemple**Veille sécurité OWASP 2024 :**

- **A01 - Broken Access Control** : Nouveaux patterns d'attaque
- **A02 - Cryptographic Failures** : Vulnérabilités des algorithmes
- **A03 - Injection** : Évolution des techniques d'injection
- **A04 - Insecure Design** : Risques de conception
- **A05 - Security Misconfiguration** : Configurations par défaut

Exemple de vulnérabilité suivie :

```
CVE-2024-1234: Vulnerability in Express.js
+-- Severity: HIGH (CVSS 7.5)
+-- Description: Prototype pollution in req.query
+-- Affected versions: < 4.18.3
+-- Impact: Remote code execution possible
+-- Mitigation: Update to Express 4.18.3+
+-- Status: Fixed in project (v4.18.5)
```

Mesures de sécurité appliquées :

- **Dépendances** : Audit automatique avec npm audit
- **Conteneurs** : Scan de vulnérabilités avec Trivy
- **Code** : Analyse statique avec SonarQube
- **Runtime** : Monitoring des anomalies avec Prometheus
- **Formation** : Sessions sécurité trimestrielles

À FAIRE / À VÉRIFIER

- Surveiller les CVE et advisories de sécurité
- Automatiser l'audit des dépendances
- Implémenter des tests de sécurité automatisés
- Former l'équipe aux bonnes pratiques sécurité
- Documenter les incidents et les contre-mesures

Contrôles Jury CDA

- Comment surveillez-vous les vulnérabilités ?
- Avez-vous automatisé l'audit de sécurité ?
- Comment gérez-vous les vulnérabilités critiques ?
- L'équipe est-elle formée à la sécurité ?
- Avez-vous un plan de réponse aux incidents ?

9.3 Application au projet

La veille technologique et sécurité influence directement les choix d'architecture et d'implémentation du projet. Les nouvelles fonctionnalités sont évaluées selon leur impact sur la sécurité, les performances, et la maintenabilité. Les mises à jour sont planifiées selon un calendrier de migration structuré.

L'intégration des bonnes pratiques découvertes améliore continuellement la qualité du code et la sécurité de l'application. La documentation des décisions techniques facilite la transmission des connaissances et la maintenance future.

Exemple**Évolution technique du projet :**

- **Q1 2024** : Migration vers React 18 pour les performances
- **Q2 2024** : Implémentation des Server Components
- **Q3 2024** : Mise à jour PostgreSQL 16 pour les JSON
- **Q4 2024** : Migration vers Node.js 20 LTS

Améliorations sécurité appliquées :

```
1 // AVANT : Validation basique
2 const validateUser = (userData) => {
3   if (userData.email && userData.password) {
4     return true;
5   }
6   return false;
7 };
8
9 // APRÈS : Validation robuste avec sanitisation
10 const validateUser = (userData) => {
11   const schema = Joi.object({
12     email: Joi.string().email().max(255).required(),
13     password: Joi.string().min(8).pattern(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d
14       )/).required(),
15     name: Joi.string().max(100).sanitize().required()
16   });
17
18   const { error, value } = schema.validate(userData);
19   if (error) {
20     throw new ValidationError(error.details[0].message);
21   }
22
23   return value;
24 };
```

Métriques d'amélioration :

Aspect	Avant	Après	Amélioration
Temps de réponse	800ms	450ms	-44%
Vulnérabilités	12	0	-100%
Couverture tests	65%	85%	+31%
Bundle size	2.1MB	1.4MB	-33%

À FAIRE / À VÉRIFIER

- Intégrer les bonnes pratiques découvertes en veille
- Planifier les migrations technologiques
- Mesurer l'impact des améliorations
- Documenter les décisions techniques
- Partager les connaissances avec l'équipe

Contrôles Jury CDA

- Comment appliquez-vous votre veille au projet ?
- Avez-vous mesuré l'impact des améliorations ?
- Vos décisions techniques sont-elles documentées ?
- Comment partagez-vous vos connaissances ?
- Votre veille influence-t-elle la roadmap ?

9.4 Liens utiles

- InfoQ : <https://www.infoq.com/>
- OWASP News : <https://owasp.org/news/>
- PostgreSQL Release Notes : <https://www.postgresql.org/docs/release/>
- React Blog : <https://react.dev/blog>
- Node.js Releases : <https://nodejs.org/en/about/releases/>

Chapitre 10

Bilan et retour d'expérience (REX)

10.1 Objectifs atteints et non atteints

L'analyse des objectifs initiaux révèle un taux d'atteinte de 85% des objectifs SMART définis. Les objectifs métier ont été largement atteints avec la livraison du MVP dans les délais. Les objectifs techniques ont été partiellement atteints, avec quelques ajustements nécessaires pour optimiser les performances. Les objectifs pédagogiques ont été dépassés grâce aux apprentissages supplémentaires acquis.

Les objectifs non atteints concernent principalement des fonctionnalités avancées reportées en v2.0 pour respecter les contraintes temporelles. Cette priorisation a permis de livrer un produit fonctionnel et stable dans les délais impartis.

Exemple

Bilan des objectifs SMART :

Objectif	Statut	Mesure	Commentaire
Réduction temps reporting	✓Atteint	-42%	Dépassé l'objectif de -40%
Livraison MVP 6 mois	✓Atteint	5.5 mois	Livré en avance
Adoption utilisateurs	Partiel	78%	Objectif 90%, formation nécessaire
Performance P95 < 500ms	✓Atteint	320ms	Dépassé l'objectif
Sécurité 0 vulnérabilité	✓Atteint	0	Objectif atteint

Objectifs non atteints :

- **Analytics avancées** : Reporté en v2.0 (complexité technique)
- **Intégrations externes** : Reporté en v2.0 (priorités métier)
- **Mobile native** : Reporté en v2.0 (PWA suffisant)
- **IA prédictive** : Reporté en v2.0 (ROI incertain)

À FAIRE / À VÉRIFIER

- Analyser objectivement l'atteinte des objectifs
- Identifier les causes des non-atteintes
- Documenter les ajustements nécessaires
- Prévoir les actions correctives pour v2.0
- Communiquer les résultats aux parties prenantes

Contrôles Jury CDA

- Quels objectifs avez-vous atteints ?
- Pourquoi certains objectifs n'ont-ils pas été atteints ?
- Comment mesurez-vous le succès de votre projet ?
- Avez-vous ajusté vos objectifs en cours de projet ?
- Quels sont vos objectifs pour la v2.0 ?

10.2 Difficultés rencontrées et solutions

Les principales difficultés ont concerné l'intégration des bases de données hétérogènes, la gestion des performances sous charge, et la coordination des équipes distribuées. Chaque difficulté a été analysée pour identifier les causes racines et implémenter des solutions durables.

L'approche de résolution de problèmes a combiné l'analyse technique, la recherche de solutions existantes, et l'innovation pour des cas spécifiques. La documentation des solutions facilite la réutilisation et l'amélioration continue.

Exemple

Tableau risques ➡ mitigation ➡ résultat :

Risque	Mitigation	Résultat	Apprentissage
Performance DB	Index + cache Redis	Latence -60%	Cache stratégique
Intégration équipes	Daily standups	Communication +40%	Processus agile
Sécurité données	Chiffrement + audit	0 incident	Sécurité by design
Délais serrés	MVP + priorités	Livraison à temps	Focus sur l'essentiel
Complexité technique	Architecture simple	Maintenance facile	KISS principe

Exemple de difficulté résolue :

Problème: Latence élevée des requêtes PostgreSQL

```

+-- Symptômes
|   +-- Temps de réponse > 2s
|   +-- Timeout des requêtes complexes
|   +-- Surcharge CPU base de données
+-- Analyse
|   +-- Requêtes sans index appropriés
|   +-- Jointures sur de gros volumes
|   +-- Pas de cache applicatif
+-- Solutions implémentées
|   +-- Création d'index composites
|   +-- Optimisation des requêtes
|   +-- Mise en place de Redis cache
|   +-- Pagination des résultats
+-- Résultat
    +-- Latence réduite à 200ms
    +-- CPU base stabilisé
    +-- Expérience utilisateur améliorée
  
```

À FAIRE / À VÉRIFIER

- Documenter toutes les difficultés rencontrées
- Analyser les causes racines des problèmes
- Rechercher des solutions existantes avant d'innover
- Tester les solutions avant déploiement
- Partager les apprentissages avec l'équipe

Contrôles Jury CDA

- Quelles ont été vos principales difficultés ?
- Comment avez-vous résolu ces difficultés ?
- Avez-vous documenté vos solutions ?
- Ces difficultés étaient-elles prévisibles ?
- Comment éviterez-vous ces difficultés à l'avenir ?

10.3 Dettes techniques et apprentissages

Les dettes techniques identifiées incluent la refactorisation de certains composants React, l'optimisation des requêtes MongoDB, et l'amélioration de la couverture de tests. Ces dettes

sont documentées avec des priorités et des estimations pour faciliter la planification des futures itérations.

Les apprentissages techniques couvrent l'architecture microservices, la gestion des performances, et les bonnes pratiques de sécurité. Ces connaissances sont transférables à d'autres projets et enrichissent l'expertise de l'équipe.

Exemple

Registre des dettes techniques :

Dettes	Priorité	Effort	Impact	Planification
Refactor composants React	Moyenne	2 semaines	Maintenabilité	v1.2
Optimisation requêtes Mongo	Haute	1 semaine	Performance	v1.1
Tests E2E manquants	Haute	1 semaine	Qualité	v1.1
Documentation API	Basse	3 jours	Développement	v1.3
Migration TypeScript	Moyenne	3 semaines	Robustesse	v2.0

Apprentissages transférables :

- **Architecture** : Pattern Repository pour l'abstraction des données
- **Performance** : Stratégies de cache multi-niveaux
- **Sécurité** : Implémentation JWT avec refresh tokens
- **Tests** : Pyramide de tests avec couverture optimale
- **DevOps** : Pipeline CI/CD avec déploiement blue-green

Exemple d'apprentissage concret :

```

1 // AVANT : Gestion d'état complexe
2 const [projects, setProjects] = useState([]);
3 const [loading, setLoading] = useState(false);
4 const [error, setError] = useState(null);
5
6 // APRÈS : Hook personnalisé réutilisable
7 const useProjects = () => {
8   const [state, setState] = useState({
9     data: [],
10    loading: false,
11    error: null
12  });
13
14  const fetchProjects = useCallback(async () => {
15    setState(prev => ({ ...prev, loading: true }));
16    try {
17      const projects = await projectService.getAll();
18      setState({ data: projects, loading: false, error: null });
19    } catch (err) {
20      setState(prev => ({ ...prev, loading: false, error: err.message }));
21    }
22  }, []);
23
24  return { ...state, fetchProjects };
25 };

```

À FAIRE / À VÉRIFIER

- Identifier et documenter toutes les dettes techniques
- Prioriser les dettes selon leur impact et urgence
- Planifier la résolution des dettes dans les futures versions
- Capitaliser sur les apprentissages pour les futurs projets
- Partager les bonnes pratiques avec l'équipe

Contrôles Jury CDA

- Quelles dettes techniques avez-vous identifiées ?
- Comment priorisez-vous ces dettes ?
- Quels apprentissages tirez-vous de ce projet ?
- Ces apprentissages sont-ils transférables ?
- Comment capitalisez-vous sur ces expériences ?

10.4 Liens utiles

- Postmortems (Google SRE) : <https://sre.google/sre-book/postmortem-culture/>
- Technical Debt : <https://martinfowler.com/bliki/TechnicalDebt.html>
- Retrospectives : <https://www.atlassian.com/team-playbook/plays/retrospective>
- Lessons Learned : <https://bit.ly/lessons-learned>
- Knowledge Management : <https://bit.ly/knowledge-management>

Chapitre 11

Conclusion et remerciements

11.1 Synthèse du projet

Ce projet de développement d'une application de gestion de projets a permis de mettre en pratique les compétences acquises en alternance CDA dans un contexte professionnel concret. L'architecture 3 tiers avec React, Node.js, PostgreSQL et MongoDB a démontré sa robustesse et sa scalabilité. Les objectifs métier ont été largement atteints avec une réduction de 42% du temps de reporting et une adoption utilisateur de 78%.

La démarche méthodologique Agile a facilité la collaboration et l'adaptation aux besoins évolutifs. Les bonnes pratiques de développement, de sécurité et de déploiement ont été appliquées avec succès, garantissant la qualité et la fiabilité de la solution livrée.

Exemple

Chiffres clés du projet :

Métrique	Valeur	Objectif
Durée de développement	5.5 mois	6 mois
Couverture de code	85%	80%
Performance P95	320ms	500ms
Vulnérabilités sécurité	0	0
Adoption utilisateurs	78%	90%
Temps de reporting	-42%	-40%

Technologies maîtrisées :

- **Frontend** : React 18, TypeScript, Redux Toolkit
- **Backend** : Node.js, Express.js, Prisma ORM
- **Bases de données** : PostgreSQL, MongoDB, Redis
- **DevOps** : Docker, GitHub Actions, SonarQube
- **Sécurité** : JWT, Argon2, OWASP Top 10

À FAIRE / À VÉRIFIER

- Synthétiser les résultats quantitatifs et qualitatifs
- Mettre en avant les compétences développées
- Identifier les points forts et les axes d'amélioration
- Préparer la présentation des résultats au jury
- Documenter les apprentissages pour la suite du parcours

Contrôles Jury CDA

- Pouvez-vous résumer les résultats de votre projet ?
- Quelles compétences avez-vous développées ?
- Quels sont vos points forts et faibles ?
- Comment évaluez-vous votre progression ?
- Quels sont vos objectifs pour la suite ?

11.2 Perspectives d'évolution

Les perspectives d'évolution du projet incluent le développement de la v2.0 avec des fonctionnalités avancées : analytics prédictives, intégrations externes, et intelligence artificielle. L'architecture actuelle permet une évolution progressive sans refactoring majeur. La roadmap technique prévoit la migration vers des technologies émergentes et l'optimisation continue des performances.

L'expérience acquise sur ce projet constitue une base solide pour aborder des projets plus complexes et des responsabilités techniques élargies. Les compétences développées sont directement applicables à d'autres contextes métier et technologiques.

Exemple

Roadmap technique v2.0 :

Q1 2025: Fonctionnalités avancées

- +-- Analytics prédictives avec machine learning
- +-- Intégrations API externes (Slack, Teams)
- +-- Notifications push temps réel
- +-- Optimisation performances (P95 < 200ms)

Q2 2025: Intelligence artificielle

- +-- Assistant IA pour la gestion de projet
- +-- Recommandations automatiques
- +-- Détection d'anomalies
- +-- Chatbot support utilisateur

Q3 2025: Évolutions technologiques

- +-- Migration vers React Server Components
- +-- Mise à jour Node.js 20 LTS
- +-- PostgreSQL 16 nouvelles fonctionnalités
- +-- Monitoring avancé avec Grafana

Compétences à développer :

- **Architecture** : Microservices, Event-driven architecture
- **Cloud** : AWS/Azure, Kubernetes, Serverless
- **IA/ML** : TensorFlow, PyTorch, MLOps
- **Sécurité** : Zero Trust, DevSecOps
- **Leadership** : Architecture decision records, mentoring

À FAIRE / À VÉRIFIER

- Définir une vision claire pour l'évolution du projet
- Identifier les technologies émergentes pertinentes
- Planifier les compétences à développer
- Anticiper les besoins métier futurs
- Maintenir la veille technologique

Contrôles Jury CDA

- Quelles sont vos perspectives d'évolution ?
- Comment prévoyez-vous l'évolution technique ?
- Quelles compétences souhaitez-vous développer ?
- Comment anticipez-vous les besoins futurs ?
- Votre projet est-il évolutif ?

11.3 Remerciements

Je tiens à remercier toutes les personnes qui ont contribué à la réussite de ce projet et à mon apprentissage en alternance CDA. Ces remerciements s'adressent à l'équipe technique, aux utilisateurs métier, aux formateurs, et à tous ceux qui ont partagé leur expertise et leur temps.

L'accompagnement reçu a été déterminant dans l'acquisition des compétences techniques et méthodologiques nécessaires à la réalisation de ce projet. Ces remerciements témoignent de la reconnaissance pour l'investissement de chacun dans ma formation professionnelle.

Exemple**Remerciements personnalisés :**

- **Mon tuteur entreprise** : Pour son accompagnement technique et son expertise
- **L'équipe de développement** : Pour la collaboration et le partage de connaissances
- **Les utilisateurs métier** : Pour leurs retours constructifs et leur patience
- **Les formateurs CDA** : Pour la transmission des fondamentaux techniques
- **La communauté open source** : Pour les outils et ressources mis à disposition

Apprentissages clés :

- **Collaboration** : L'importance du travail d'équipe en développement
- **Communication** : La nécessité de bien communiquer avec les parties prenantes
- **Adaptabilité** : La capacité à s'adapter aux changements et contraintes
- **Qualité** : L'exigence de qualité dans le développement logiciel
- **Veille** : L'importance de la veille technologique continue

À FAIRE / À VÉRIFIER

- Exprimer sa gratitude de manière sincère et personnalisée
- Reconnaître l'apport spécifique de chaque personne
- Mettre en avant les apprentissages tirés des interactions
- Maintenir les relations professionnelles établies
- Préparer la suite du parcours avec confiance

Contrôles Jury CDA

- Qui souhaitez-vous remercier particulièrement ?
- Quels apprentissages tirez-vous de ces interactions ?
- Comment envisagez-vous la suite de votre parcours ?
- Quelles relations professionnelles avez-vous nouées ?
- Comment comptez-vous maintenir ces relations ?

11.4 Déploiement et documentation

Dans cette section, vous devez présenter votre stratégie de déploiement et la documentation technique de votre projet. Le jury attend une compréhension claire de votre approche

opérationnelle et de la maintenabilité de votre solution.

Votre stratégie de déploiement : *[Décrivez votre approche de déploiement et de documentation]*

11.4.1 Docker

Dans cette sous-section, vous devez détailler votre approche de containerisation avec Docker. Le jury attend une explication claire de votre Dockerfile et de votre orchestration.

Votre containerisation : *[Décrivez votre Dockerfile et votre approche Docker]*

Conteneurisation

Votre Dockerfile : *[Décrivez votre Dockerfile multi-stage]*

Exemple

Dockerfile multi-stage :

```
1 # Stage 1: Build
2 FROM node:18-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci --only=production
6 COPY . .
7 RUN npm run build
8
9 # Stage 2: Production
10 FROM node:18-alpine AS production
11 RUN addgroup -g 1001 -S nodejs
12 RUN adduser -S nextjs -u 1001
13 WORKDIR /app
14 COPY --from=builder /app/node_modules ./node_modules
15 COPY --from=builder /app/dist ./dist
16 COPY --from=builder /app/package*.json ./
17 RUN chown -R nextjs:nodejs /app
18 USER nextjs
19 EXPOSE 3000
20 ENV NODE_ENV=production
21 CMD ["node", "dist/index.js"]
```

Compose

Votre Docker Compose : *[Décrivez votre orchestration des services]*

Exemple**Docker Compose pour l'environnement complet :**

```

1 version: '3.8'
2 services:
3   app:
4     build: .
5     ports:
6       - "3000:3000"
7     environment:
8       - NODE_ENV=production
9       - DATABASE_URL=postgresql://user:pass@postgres:5432/projectdb
10    depends_on:
11      - postgres
12      - redis
13    restart: unless-stopped
14
15    postgres:
16      image: postgres:15-alpine
17      environment:
18        - POSTGRES_DB=projectdb
19        - POSTGRES_USER=user
20        - POSTGRES_PASSWORD=pass
21      volumes:
22        - postgres_data:/var/lib/postgresql/data
23      restart: unless-stopped
24
25    redis:
26      image: redis:7-alpine
27      restart: unless-stopped
28
29 volumes:
30   postgres_data:

```

11.4.2 GitHub (code source)

Dans cette sous-section, vous devez présenter votre organisation du code source sur GitHub. Le jury attend une explication claire de votre structure de repository et de vos conventions.

Votre organisation GitHub : *[Décrivez votre structure de repository et vos conventions]*

Exemple**Structure du repository :**

```

project-management-app/
+-- src/                      # Code source
|  +-- frontend/              # Application React
|  +-- backend/               # API Node.js
|  +-- shared/                # Code partagé
+-- docs/                     # Documentation
|  +-- api/                   # Documentation API
|  +-- deployment/            # Procédures de déploiement
|  +-- architecture/          # Documentation architecture
+-- scripts/                  # Scripts utilitaires
+-- tests/                    # Tests automatisés
+-- docker/                   # Configuration Docker
+-- .github/                  # GitHub Actions et templates

```

11.4.3 CI/CD

Dans cette sous-section, vous devez présenter votre pipeline CI/CD. Le jury attend une explication claire de votre automatisation et de vos environnements.

Votre pipeline CI/CD : *[Décrivez votre automatisation et vos environnements]*

Exemple

Pipeline CI/CD GitHub Actions :

```
1 name: CI/CD Pipeline
2 on:
3   push:
4     branches: [main, develop]
5   pull_request:
6     branches: [main, develop]
7
8 jobs:
9   test:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v4
13      - name: Setup Node.js
14        uses: actions/setup-node@v4
15        with:
16          node-version: '18'
17      - name: Install dependencies
18        run: npm ci
19      - name: Run tests
20        run: npm test -- --coverage
21
22    deploy-staging:
23      runs-on: ubuntu-latest
24      needs: test
25      if: github.ref == 'refs/heads/develop'
26      steps:
27        - name: Deploy to staging
28          run: ./scripts/deploy.sh staging
29
30    deploy-production:
31      runs-on: ubuntu-latest
32      needs: test
33      if: github.ref == 'refs/heads/main'
34      steps:
35        - name: Deploy to production
36          run: ./scripts/deploy.sh production
```

11.4.4 SonarQube

Dans cette sous-section, vous devez présenter votre approche de qualité du code avec SonarQube. Le jury attend une explication claire de vos métriques et de votre intégration.

Votre qualité du code : *[Décrivez vos métriques de qualité et votre intégration SonarQube]*

Exemple**Métriques de qualité SonarQube :**

Métrique	Objectif	Actuel	Statut
Couverture de code	> 80%	85%	✓
Duplication	< 3%	1.2%	✓
Complexité cyclomatique	< 10	7.3	✓
Maintenabilité	A	A	✓
Fiabilité	A	A	✓
Sécurité	A	A	✓

11.4.5 Swagger

Dans cette sous-section, vous devez présenter votre documentation API avec Swagger. Le jury attend une explication claire de votre documentation et de son utilisation.

Votre documentation API : *[Décrivez votre documentation Swagger et son utilisation]*

Exemple**Documentation API Swagger :**

```
1 openapi: 3.0.0
2 info:
3   title: Project Management API
4   version: 1.0.0
5   description: API pour la gestion des projets
6
7 paths:
8   /projects:
9     get:
10      summary: Liste des projets
11      responses:
12        '200':
13          description: Liste des projets
14          content:
15            application/json:
16              schema:
17                type: object
18                properties:
19                  data:
20                    type: array
21                    items:
22                      $ref: '#/components/schemas/Project'
23
24 components:
25   schemas:
26     Project:
27       type: object
28       properties:
29         id:
30           type: string
31           format: uuid
32         name:
33           type: string
34         description:
35           type: string
36         createdAt:
37           type: string
38           format: date-time
```

À FAIRE / À VÉRIFIER

- Documenter complètement votre API avec Swagger
- Intégrer SonarQube dans votre pipeline CI/CD
- Organiser votre code source de manière claire
- Automatiser tous les aspects du déploiement
- Maintenir la documentation à jour

Contrôles Jury CDA

- Comment organisez-vous votre code source ?
- Votre pipeline CI/CD est-il complet ?
- Comment mesurez-vous la qualité de votre code ?
- Votre API est-elle documentée ?
- Comment gérez-vous les déploiements ?

11.5 Liens utiles

- Dockerfile reference : <https://docs.docker.com/reference/dockerfile/>
- Docker Compose : <https://docs.docker.com/compose/>
- GitHub Actions : <https://docs.github.com/actions>
- SonarQube : <https://docs.sonarsource.com/sonarqube/latest/>
- Swagger/OpenAPI : <https://swagger.io/specification/>
- CDA Formation : <https://www.cda.asso.fr/>
- Colint.school : <https://colint.school/>